

**Real-Time Kinematic Simulation
of Planar Linkage Mechanisms**

A Thesis presented

by

Zhijie Lyu

to

The Graduate School

in Partial Fulfillment of the

Requirements

for the Degree of

Master of Science

in

Mechanical Engineering

Stony Brook University

August 2022

Abstract of the Thesis

**Real-Time Kinematic Simulation
of Planar Linkage Mechanisms**

by

Zhijie Lyu

Master of Science

in

Mechanical Engineering

Stony Brook University

2022

This thesis is the documentation for the planar kinematic simulation algorithm for MotionGen's latest version. In this implementation, the mobility-based analysis is used to extract the solution procedure, which decomposes a system of equations to reduce the simulation time. Furthermore, we update the two types of solvers for kinematic constraint systems, which allows actuators to be placed anywhere in the kinematic structure; a trick is used to reinterpret the prismatic joints so that all constraint equations in the system are either distance constraint or angle constraint; we also set up a unified representation of kinematic structures in this version, which allows us to implement graphical analysis techniques.

Table of Contents

1	Introduction	1
1.1	Background	1
1.2	Available Techniques	2
1.3	Implementation Specifications	4
1.4	Overview of Content	5
2	Problem Formulation and Solvers	6
2.1	Geometric Constraint Equations	7
2.2	Arc Intersection	9
2.3	Newton's method in optimization	11
2.3.1	System of Equations Formulation for NR-solver	12
2.3.2	Derivation for NR-solver	14
2.3.3	NR-solver Algorithm and its Analysis	18
2.3.4	System of Equations Formulation for GN-solver	20
2.4	Removal of the Forward Chain Solver	22
2.5	Conclusion	23
3	Arrangement of Mechanical Components	24
3.1	Basic Components of a Linkage Mechanism	24
3.2	Representation of a Planar Linkage using Matrices	25
3.3	Reinterpretation of Prismatic(P) Joint	31
3.4	Operations on Static Components	35
3.5	Interpretation of Actuators	43
4	Mobility-based Analysis	45
4.1	Usage of the Mobility Formula	46
4.2	Graph Traversal Algorithms	50
4.3	Overall Algorithm for Simulation	54

4.4	Case analysis	56
4.4.1	Floating Actuator	57
4.4.2	Multi-Slot	58
4.4.3	Extremely Complex Structure	60
4.4.4	Extremely Large Number of States	62
4.5	Structural Complexity Estimation	62
5	Conclusion and Future Work	65

List of Figures

1.1	This is the figure of content.	5
2.1	This is an R robot arm mechanism.	8
2.2	This figure shows the intersection of circles. There is an imaginary link between j_p and j_q when the two joints are solved, and it will be used if necessary.	10
2.3	This is a mechanism that cannot be solved with the ArcSect solver only. To solve this mechanism, j_1, j_3, j_6 needs to be solved altogether.	12
3.1	This is a common four-bar mechanism. The joints are marked with numbers in black and the links are marked with numbers in red.	27
3.2	A RRRP mechanism	31
3.3	This is the reinterpreted RRRP mechanism for Figure 3.2. A joint is in the far bottom-right.	31
3.4	All nine combinations of RR, PR, RP, and PP dyads are displayed above. The first row shows RRRR, RRRP, and RRPR mechanisms. The second row shows RRPP, RPPR, and RPRP mechanisms. The third row shows PRRP, RPPP, and PRPP mechanisms. All nine mechanisms can be simulated.	34
3.5	The two ternary links (green and blue) share two joints, meaning these two links are rigidly connected and can be seen as a 4-ary link (red).	38
3.6	The letters denote the joints. For the rotary actuator (left), $l_{i,k}$ is the corresponding dynamic link. For the linear actuator (right), $l_{i,j}$ is the corresponding dynamic link.	43

4.1	There are 7 rigid bodies and 7 rotary joints in this mechanism. One joint connects three rigid bodies while other joints connect two joints. The value of dof is $(7 - 1) \cdot 3 - (6 \cdot (2 - 1) + (3 - 1) \cdot 1) \cdot 2 = 2$. There are two actuators here, which together remove all remaining DOF. Therefore, this kinematic chain is properly constrained.	47
4.2	The mechanisms on the left and the middle can be converted to the mechanism on the right based on two different rules to merge links. The mechanism on the left will first merge l_{13} , l_{14} , l_{16} because the DOF for these three links is zero. Then this new link is merged with l_{12} and l_{15} because the DOF is again zero. For the mechanism in the middle, because l_7 and l_8 share two joints, these two links are merged into one. The resultant mechanism for the left and the middle ones is the one on the right. Therefore, these three mechanisms are all RRRR four-bar linkage mechanisms with two coupler points on the moving frame.	48
4.3	Left: over-constrained kinematic chain ($dof = (6 - 1) \cdot 3 - (4 \cdot (3 - 1)) \cdot 2 = -1$). Right: under-constrained kinematic chain ($dof = (4 - 1) \cdot 3 - (4 \cdot (2 - 1)) \cdot 2 = 1$)	49
4.4	Partly over-constrained and partly under-constrained kinematic chain. ($dof = (9 - 1) \cdot 3 - ((4 - 1) \cdot 2 + (3 - 1) \cdot 2 + (2 - 1) \cdot 2) \cdot 2 = 0$)	49
4.5	Left: RRRR mechanism whose actuator is fixed to ground; Right: RRRR mechanism whose actuator is fixed to a moving joint.	57
4.6	Left: The multi-slot mechanism, Right: the interpretation of such a mechanism. The constraining joints and constraining links are not displayed for visual clearness.	59
4.7	This millipede-like mechanism can only be solved by an optimization method and all movable joints have to be solved all at once. $l_1 \sim l_5$, l_6 together with the actuator and link l_{25} , and $l_7 \sim l_{12}$ are the “leg” links. Link $l_{13} \sim l_{24}$ are the “body” links. Link l_{26} is the ground link.	60
4.8	One of the static links of the millipede structure is on the four-bar linkage mechanism.	61

4.9	This is an equivalent representation for the mechanism for Figure 4.8. The binary links colored in green form a typical four-bar linkage mechanism and can be solved according to a similar pattern in Section 4.4.1.	61
4.10	A pantograph machine	63

List of Tables

2.1	Solver comparison table	23
3.1	This is the adjacency list format of storage for Figure 3.3 in link major order. Compared to the B matrix in Equation 3.23, the exclusion of joints for a particular link is expressed through not saving it in the list.	37
3.2	This is the adjacency format in joint major order for Figure 3.3.	37
4.1	Table of statics of mechanisms	64

Acknowledgements

I would like to thank my parents and my uncle and aunt for their constant support and encouragement. I am very grateful to have Professor Purwar as my advisor. His mentorship, continuous support and caring attitude kept me going. It is an honor working with him and I truly admire his working style.

I would also like to thank MotionGen main developer Wei Liao and Ex-MotionGen developer Prerna Kothari for their dedication to the development and documentation of MotionGen. Furthermore, I'd like to thank my lab colleagues Anar Nurizada, Xueting Deng, Digvijay, who gave me valuable advice, making the development an interesting journey.

I would like to express my thanks to the committee members, Professor Qiaode Jeffrey Ge and Professor Nilanjan Chakraborty, for their precious time and their presence on the thesis committee.

This work is financially supported by the National Science Foundation.

Chapter 1

Introduction

This thesis is the documentation for the planar kinematic simulation algorithm for MotionGen’s latest version. In this implementation, the mobility-based analysis is used to extract the solution procedure, which decomposes a system of equations to reduce the simulation time. Furthermore, we update the two types of solvers for kinematic constraint systems, which allows actuators to be placed anywhere in the kinematic structure; a trick is used to reinterpret the prismatic joints so that all constraint equations in the system are either distance constraint or angle constraint; we also set up a unified representation of kinematic structures in this version, which allows us to implement graphical analysis techniques.

1.1 Background

Kinematic simulation of a multi-body system is an important part of CAD (*Computer Aided Design*). CAD tools with simulation capability allow one to predict the kinematic behavior of multi-body systems. From the first design concept stage to the final prototype stage, the CAD tools are excellent companions, providing a sufficient amount of data and visual feedback which allows the engineers to study the influence of the configuration of a particular mechanical component in a certain mechanism.

A faster simulation response can significantly reduce the time cost because designing a product is an iterative process. In other words, a shorter simulation procedure helps improve design efficiency. Furthermore, while the capability of computer systems has increased tremendously in recent

years and has become affordable to a broader group of people, the power in different computer systems dramatically differs. Recently, mobile devices and web-based apps have been popular choices used by college students in their studying process. Therefore, a time-efficient computational structure is desperately needed to allow more electronic devices to handle kinematic simulation.

Planar linkage mechanisms are commonly used in workshops due to their simplicity, practicality, and performance characteristics, whose kinematic analysis and synthesis have been done by stalwarts like McCarthy and Soh [1], Sandor and Erdman [2], Hunt [3], Hartenberg and Denavit [4], Suh and Radcliffe [5]. To address the demand for designing and evaluating these mechanisms, our lab presented MotionGen, a web-based kinematic design package, in 2012. The newest version is more efficient in solving the planar linkage kinematic simulation problem and provides more design flexibility than previous iterations.

Three major enhancements for the planar linkage kinematic simulation have been introduced since the creation of MotionGen. First, Lodha, A. [6] introduced an optimization-based algorithm into this application to simulate complex mechanisms, with revolute(R) and prismatic(P) joints taken into account. Then, Kothari, P. [7] introduced the reinterpretation of prismatic(P) joints as revolute(R) joints to uniformly decide the geometric errors that are brought by both types of joints. Finally, this latest version was developed by Wei Liao and the author, who introduced a refined geometrical constraint expression, an easy-to-understand data structure, and a graph traversal algorithm that extracts the solution procedure.

1.2 Available Techniques

For kinematic simulation of planar linkages, the focus is solving the position of points in interest (POI), among which is a set of geometric constraints. Then, the velocity and acceleration analyses can come after all the positions in all states are solved. For planar linkages, these POIs are the joint positions. The available methods to wholly or partly solve a mechanism, in other words, the *solvers*, can be generally classified into two categories: analytical and numerical. Analytical methods involve solving nonlinear loop-closure equations. Numerical methods can solve all positional problems of any degree of complexity and number of elements. Generally, the solvers need to

solve the same set of joint positions for each input to get each corresponding state.

One of the simplest analytical methods for planar linkages is the dyadic decomposition [2, 8]. Based on the identification of four-bar loops in the mechanism and the positioning of one of the dyads, this method solves the other dyad with arc intersection. This method applies to linkages of low complexity, and it is not valid when 1) the input is not in the four-bar loop and 2) the mechanism itself is of high complexity.

For complex structures, researchers have developed different methods. Dhingra et al. [9] follow the computation of a Gröbner basis with a solution based on a Sylvester-type determinant and use symbolic computation extensively. Wampler [10, 11] proposed a combination of complex plane formulation and Dixon determinant procedure from Nielsen and Roth [12]. He presents the formulation of kinematic equations in complex planes using isotropic coordinates, which produces loop equations in a simpler form that can be solved using the Sylvester-type formula. This method aims to find all the roots of the equations, so finding the solutions for a Stephenson 6-bar is equivalent to finding the eigenvalues of a 6×6 matrix or roots of a degree 6 equation. Hernández et al. [13] have proposed a ‘Geometric Iterative Technique’ to solve the position problem with only revolute joints. This is a novel approach to solving the position problem, where an initial guess is not required, as in the case of local-optimization methods. However, it is common to resort to the Newton–Raphson (NR) method to solve systems of nonlinear equations [14]. The NR method is prevalent because this method tries to find a particular solution numerically and has quadratic convergence in the neighborhood of the solution.

Additionally, the solvers can be arranged to solve different parts of a mechanism. Specifically, this arrangement is achieved through the graphical identification of the solvable substructure, whose solvability is based on known information. In other words, for planar linkage mechanisms, the unknown joints in the solvable substructure are solved with a particular solver based on a specific set of geometric constraints and the already-solved joints (known joints). Then, these unknown joints in the solvable substructure are solved and become known. Sequentially, in the same manner, another substructure that contains some or whole of the remaining unsolved joints can be identified and solved. As a result, a particular state/set of joint positions is solved.

Such an identification process is related to the graphical properties of the

kinematic structure of the mechanism regardless of the specific joint positions or input. Therefore, the identification process needs to be done only once for the specific mechanism. The identification process involves graph traversal, through which the *extraction of the solving procedure* is found. Furthermore, if a structure can be decomposed into several pieces and solved part by part, it usually takes less time than solving the entire structure in one single step. Lots of mathematical rigorous studies have been done by Ait-Aoudia et al. [15, 16], Bouma [17], and Fudos and Hoffmann [18].

1.3 Implementation Specifications

We make a set of requirements for our latest implementation. Specifically, the algorithm should solve kinematic structures:

1. of any combination of revolute and prismatic joints,
2. of any mobility and any number of input (linear and rotary), and
3. of any type of actuation profile.

Next, the extraction of the solving procedure part of the algorithm should identify closed-loops from kinematic structures. Furthermore, the algorithm should uniformly represent the kinematic structure with a set of matrices.

Additionally, our implementation should be easy to use for different groups of people, from elementary to college students and beyond, which means that an intuitive and interactive GUI should be developed. Last but not least, our implementation should be easy to access, which means that the implementation should be web-based and cross-platform.

There are simulation packages in the market like Linkage [19], PMKS/PMKS+ [20, 21], SAM [22], GIM [23], and Mekin2d [24]. Generally, these simulation packages are difficult for newcomers. Linkage has limited simulation capabilities and cannot solve mechanisms with complex structures. Although it can be accessed online, PMKS/PMKS+ has an unintuitive UI and does not allow floating actuators. SAM, GIM, and Mekin2d must be downloaded and run on a desktop computer.

Comparatively, this latest version of MotionGen has adopted two kinds of solvers and a graph traversal algorithm, which together can solve the structure efficiently. Specifically, this algorithm can satisfy all the simulation

algorithm requirements mentioned above. Furthermore, MotionGen is a web-based application whose UI is redeveloped to allow more straightforward creations of mechanical components.

1.4 Overview of Content

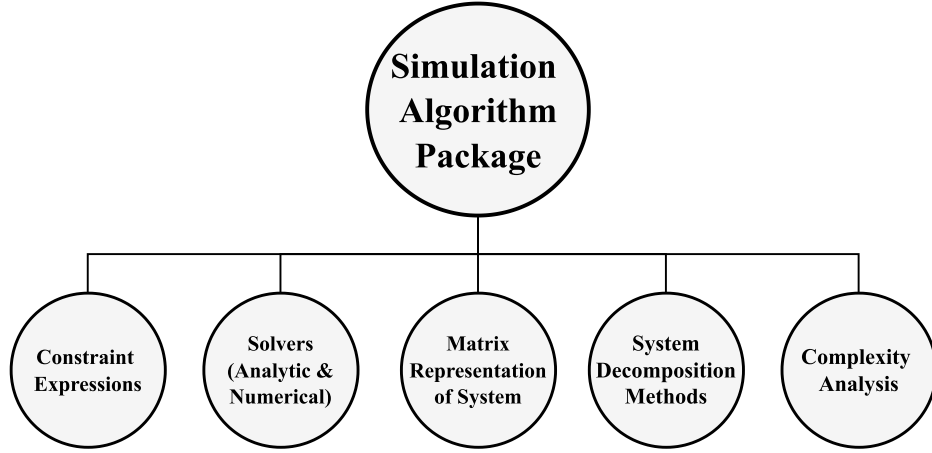


Figure 1.1: This is the figure of content.

This simulation algorithm package contains different techniques shown in Figure 1.1. First, we need to list the geometric relationships that need to be satisfied, following which are the solvers (numerical and analytic) to look for the solution that satisfies all these constraints. Then, to decompose the system and reduce simulation time, we use matrix representation of the kinematic structures and implement the decomposition method that takes these matrices as input. After the decomposition, the extraction of the solving procedure is found, with which the complexity analysis for both time and kinematic structure can be done. Furthermore, the decomposed system can be handled by the solvers more efficiently.

The remaining parts of this thesis are arranged as follows. First, in Chapter 2, the formulation of a planar linkage simulation problem and the solvers are presented. Next, Chapter 3 introduces the arrangement and interpretation of mechanical components. Chapter 4 introduces the graph traversal algorithm to search for an efficient solving process. Finally, in Chapter 5, a summary of the simulation algorithm and possible future work are presented.

Chapter 2

Problem Formulation and Solvers

Kinematic simulation of planar linkage mechanisms with n links is equivalent to solving a system of equations, which describes the geometric constraints from 1) the kinematic structure itself and 2) the set of input from the actuators. Then, because the kinematic simulation is a finite displacement problem, the set of joint positions is obtained after this system of equations is solved. Furthermore, such systems may have multiple solutions that map to different circuits or branches. Typically while designing, only one particular solution is desired. When there is more than one solution, only one is picked according to the initial condition.

Since Lodha, A. [6], two types of solvers have been introduced into MotionGen, which are analytical and numerical-based methods. The analytical methods include forward kinematic chain and arc intersection, and the optimization-based method is the Newton Raphson (NR) method. Both types have been used in the simulation package in the interest of both saving times and handling the mechanisms with complex structures. The arc intersection method is very time-efficient, while it cannot solve mechanisms with complex loops. On the other hand, the NR-solver can handle mechanisms with whatever complexity while its time to compute, or its *time complexity*, is much more significant than arc intersection as the number of joints increases. In this iteration, the forward kinematic chain solver has been removed, and another sometimes more efficient Newton's method is introduced. We will discuss these two Newton's methods in this chapter in a chronological manner, i.e., we will first discuss the NR solver, and the new Newton's solver will

come later.

Therefore, the contents in this chapter are arranged as follows: Section 2.1 introduces the two kinds of constraints for all planar linkage mechanisms, where one type of constraint is interpreted differently compared to the last iteration [7]. Section 2.2 will introduce the modified arc intersection (ArcSect) solver. Section 2.3 will first go through the modified NR-solver according to the new interpretation of constraints, and then Section 2.3.4 will introduce another Newton’s method, the Gauss-Newton (GN) solver, which is sometimes more efficient than the NR-solver. Finally, Section 2.4 will briefly go through the removed solver and give a comprehensive comparison of all the solvers mentioned throughout this chapter.

Additionally, because all prismatic(P) joints can be reinterpreted into a combination of revolute(R) joints and RR binary links (two sides of the link are revolute(R) joints), there is not a specific model for prismatic(P) joints in the simulation solver aspect. Hence, it does not show up in this chapter. Further details regarding the reinterpretation of prismatic joints will be introduced in Chapter 3.

2.1 Geometric Constraint Equations

Generally, a solver is to handle a sub- or whole kinematic structure, inside which there is a proper system of equations corresponding to geometric constraints. There are two types of constraints for a revolute joint system, which are 1) the distance constraint between two joints and 2) the angle constraint between two rigid bodies.

Each distance constraint corresponds to a *binary link* (two joints), and each angle constraint corresponds to two connected binary links (three joints). Figure 2.1 shows a simple combination of these two constraints. In this figure, the moving binary link (green) has one joint (j_0) snapped to the ground link (gray), and its other joint (j_2) is actuated by the actuator (light red), whose path is drawn in blue. The distance constraint takes two joint positions into account. The angle between the moving link and the ground link is determined by three joints because at least two joints are needed to define a rigid body and these two rigid bodies share one joint.

A planar point has two degrees of freedom. In other words, two variables need to be solved together for a particular joint position. Each of the two abovementioned constraints for this figure contributes one constraint equa-

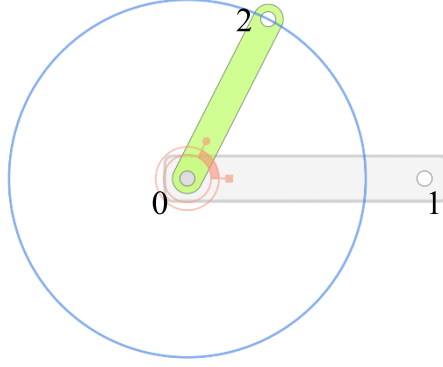


Figure 2.1: This is an R robot arm mechanism.

tion. Therefore, the joint position can be solved. For each angle caused by the actuator, there is a solution for j_2 . When the solutions for the consecutive angles are connected, the path of j_2 (blue) is obtained.

The distance constraint equation can be written as,

$$f_1 = (\sqrt{(x_2 - x_0)^2 + (y_2 - y_0)^2} - l_{0,2}) \quad (2.1)$$

x_0, y_0, x_2, y_2 are the positions of j_0, j_2 in x, y dimensions respectively. $l_{0,2}$ is the unchanged distance between these two joints, which also denotes the binary link itself. f_1 is the *error function* that denotes the first distance constraint and this distance constraint is satisfied only when $f_1 = 0$.

Then, the angle constraint equation can be written as,

$$g_1 = (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0) - l_{1,0} \cdot l_{0,2} \cdot \sin(\alpha_{1,0,2}) \quad (2.2)$$

g_1 is the *error function* that denotes the 1st angle constraint and this angle constraint is satisfied when $g_1 = 0$. $\alpha_{1,0,2}$ is the angle between vector $v_{0,1}$ and $v_{0,2}$, whose value is decided by the actuator $\alpha_{1,0,2}$. Unlike distance constraints, an angle constraint must be represented by three joints in a particular order. This is because we use the cross-product value of two vectors to represent the angle input of the actuator uniquely. Equation 2.3 denotes two equivalent expressions to compute a general cross-product value. This representation is more reasonable than directly computing the error between the actual and desired angles because only one trigonometric function is used.

$$\begin{aligned} \text{norm}(\vec{v}_{q,p} \wedge \vec{v}_{q,r}) &= l_{p,q} \cdot l_{q,r} \cdot \sin(\alpha_{p,q,r}) \\ &= (x_p - x_q)(y_r - y_q) - (x_r - x_q)(y_p - y_q) \end{aligned} \quad (2.3)$$

Furthermore, because the angle constraint was not implemented into arc intersection or the Newton methods in the previous MotionGen, Section 2.2 and Section 2.3 will introduce how the two solvers handle this type of constraint, respectively.

In conclusion to this section, not to lose generality, the i^{th} error function for distance constraint between j_p, j_q can be written as,

$$f_i = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} - l_{p,q} \quad (2.4)$$

Similarly, the j^{th} error function for the angle constraint between vector $\vec{v}_{q,p}, \vec{v}_{q,r}$ can be written as,

$$g_j = (x_p - x_q)(y_r - y_q) - (x_r - x_q)(y_p - y_q) - l_{p,q} \cdot l_{q,r} \cdot \sin(\alpha_{p,q,r}) \quad (2.5)$$

2.2 Arc Intersection

The arc intersection (ArcSect) solver uses a graph-analytical approach, which handles the special situation when two of the three joints are already solved and the unknown joint connects to the other two joints respectively through binary RR links.

To illustrate, suppose we have j_p, j_q solved and j_r unsolved but we also have two binary links $l_{p,r}$ and $l_{q,r}$, i.e., two distance constraints. Then, the solution of j_r is at the intersection of two circles. One of the circle's centers is j_p and its radius is $l_{p,r}$. The center of the other circle is j_q , and its radius is $l_{q,r}$. j_r is on the intersection of these two circles because only the intersection point fits both distance constraints.

Arc intersection can have no solution, one (repeated) solution, and two solutions, which are handled differently.

1. If there is no solution, then this position cannot be achieved, and the input cannot be satisfied. Therefore, the simulation terminates.
2. If there is one solution, the two centers and the intersection point are on one straight line, which can be a singularity configuration. It is a valid solution, and there is no need to choose the solution.

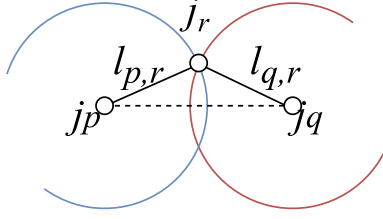


Figure 2.2: This figure shows the intersection of circles. There is an imaginary link between j_p and j_q when the two joints are solved, and it will be used if necessary.

3. If there are two solutions, then which solution to pick depends on the initial conditions. The solution tends to be closer to the sum of the previous position and the estimated velocity. The estimated velocity is used to help the solution run away from the singularity configuration. For the k^{th} position, if the solution is the closest to the *reference point* expressed below, then it will be picked.

$$\mathbf{p}_{k,ref} = \mathbf{p}_{k-1} + (\mathbf{p}_{k-1} - \mathbf{p}_{k-2})\Delta T \quad (2.6)$$

ΔT is a time factor. We set it to 1 by default.

Additionally, if there is a rotary actuator inside these three joints, the vector cross-product value is always computed and examined, regardless of which joint the actuator is placed on, and the reference point will not be used. That means, based on the placement of the actuator, the angle constraint errors for the two solutions are computed according to the format in Equation 2.5, and the solution with the least angle constraint error will be chosen. Furthermore, we set a threshold for the error, which is a small value because the solution is analytical. If the error exceeds the threshold, then the simulation also terminates.

The mechanism in Figure 2.1 can be solved with this solver. But before that, one new concept needs to be introduced.

Dynamic Link: A dynamic link is an imaginary link that can be used for algebra. It is dynamic because the link length can change according to its corresponding actuator. Details about the dynamic links will be presented in Chapter 3.

For the case in Figure 2.1, the solving procedure is as follows.

1. Compute dynamic link length $l_{1,2}$ according to the law of cosine. In this case, that would be

$$l_{1,2} = \sqrt{l_{0,1}^2 + l_{0,2}^2 - 2 \cdot l_{0,1} \cdot l_{0,2} \cos(\angle v_{0,1}, v_{0,2})} \quad (2.7)$$

The cosine value is provided by the input angle of the actuator.

2. Solve the arc intersection of the two circles. The center of the first circle is j_0 and its radius is $l_{0,2}$. The center of the other circle is j_1 and its radius is $l_{1,2}$.
3. Two solutions are found and there is an actuator among the three joints. Compute the angle error function 2.5 for the two results. Pick the one that leads to a smaller absolute value.

The advantage of this method is that solving one joint requires a time of $O(1)$. Hence, if a mechanism with n number of joints and all these joints can be solved using this method, the time complexity is $O(n)$. However, its drawback is that it cannot handle complex structures. For example, there are no four-bar loops that contain the actuator inside the mechanism shown in Figure 2.3. To address this problem, another type of solver, the optimization-based solver, is introduced.

2.3 Newton's method in optimization

Throughout the development of the current simulation code, two main Newton-based algorithms are developed. Both methods can solve the mechanism of whatever complexity. Previous implementations have already introduced the Newton-Raphson (NR) solver, but did not introduce the angle constraints into the solver. Then, through our experiment in Newton's optimization methods, the Gauss-Newton (GN) solver provides a faster simulation convergence for common cases. Therefore, this section will first introduce the formulation of the system of equations for the modified NR-solver to handle both distance and angle constraints. Next, because the NR-solver is developed first and the two methods share some elements in common, the GN-solver is placed in the latter part of this section.

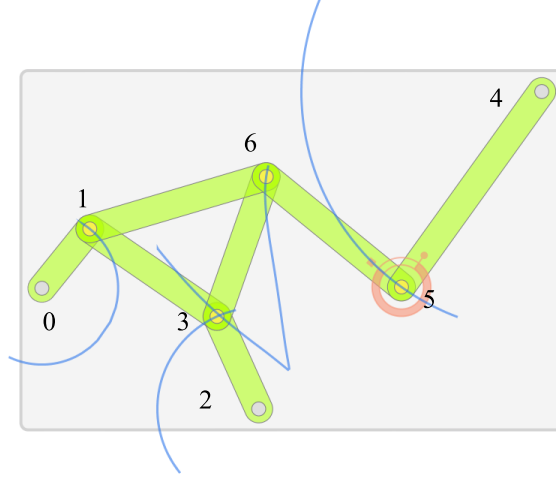


Figure 2.3: This is a mechanism that cannot be solved with the ArcSect solver only. To solve this mechanism, j_1, j_3, j_6 needs to be solved altogether.

2.3.1 System of Equations Formulation for NR-solver

Without going into the details of finding the solving procedure, the mechanism in Figure 2.3 can be solved according to the steps below.

1. Dynamic link $l_{4,6}$ is computed according to the law of cosine and the rotary actuator.
2. j_0, j_2, j_4 are marked as solved because they are fixed joints.
3. j_1, j_3, j_6 are solved altogether through the NR-solver. The solution is picked from the last result of NR-optimization.
4. j_5 is solved through the ArcSect solver. The center of the first circle is j_4 and its radius is $l_{4,5}$. The center of the other circle is j_6 and its radius is $l_{6,5}$. The solution is picked according to the error function 2.4 between the solution and the estimated vector cross product value of $v_{5,6}$ and $v_{5,4}$.

For the third step mentioned above, the system of equations to solve is,

$$\begin{cases} f_1 &= (\sqrt{(x_0 - x_1)^2 + (y_0 - y_1)^2} - l_{0,1}) \\ f_2 &= (\sqrt{(x_1 - x_3)^2 + (y_1 - y_3)^2} - l_{1,3}) \\ f_3 &= (\sqrt{(x_2 - x_3)^2 + (y_2 - y_3)^2} - l_{2,3}) \\ f_4 &= (\sqrt{(x_1 - x_6)^2 + (y_1 - y_6)^2} - l_{1,6}) \\ f_5 &= (\sqrt{(x_3 - x_6)^2 + (y_3 - y_6)^2} - l_{3,6}) \\ f_6 &= (\sqrt{(x_4 - x_6)^2 + (y_4 - y_6)^2} - l_{4,6}) \end{cases} \quad (2.8)$$

The set of equations in 2.8 is a set of distance constraints for six variables $(x_1, y_1, x_3, y_3, x_6, y_6)$. However, for some very complex mechanisms, the actuators also need to be taken into account. But now, let us only consider the case in Figure 2.3. If j_1, j_3, j_5 , and j_6 are solved altogether, besides $f_1 \sim f_5$, the resultant system of equations contains three more constraint equations, which are

$$\begin{cases} f_7 &= (\sqrt{(x_4 - x_5)^2 + (y_4 - y_5)^2} - l_{4,5}) \\ f_8 &= (\sqrt{(x_6 - x_5)^2 + (y_6 - y_5)^2} - l_{6,5}) \\ g_1 &= ((x_6 - x_5)(y_4 - y_5) - (x_4 - x_5)(y_6 - y_5) - l_{6,5} \cdot d_{5,4} \cdot \sin(\alpha_{6,5,4})) \end{cases} \quad (2.9)$$

As indicated in Step 4, Equation 2.9 can be solved with the ArcSect solver. However, all these functions can also be solved altogether with the NR-solver (It is just less efficient). As a result, this problem is formulated as, find the set of values of $x_1, y_1, x_3, y_3, x_5, y_5, x_6, y_6$ that minimizes

$$F = \sum_{i=1}^8 f_i^2 + \sum_{j=1}^1 g_j^2 \quad (2.10)$$

F is the *objective function* for this mechanism, whose value is equal to the sum of squared error. More generally, the piled-up optimization objective function which takes l numbers of binary links and a numbers of actuators into account, can be written as,

$$F(\mathbf{x}) = \sum_{i=1}^l f_i^2(\mathbf{x}) + \sum_{j=1}^a g_j^2(\mathbf{x}) \quad (2.11)$$

The column vector $\mathbf{x}$ collects all positional variables. If there are n number of joints in this function, then $\mathbf{x}$ can be written as,

$$\mathbf{x} = \begin{bmatrix} x_p \\ y_p \\ x_q \\ y_q \\ \vdots \\ x_r \\ y_r \end{bmatrix}_{2n \times 1} \quad (2.12)$$

When $F(\mathbf{x})$ is zero or close to zero, all constraints are satisfied. In other words, the set of joint positions is found. In MotionGen, for each step, the set of joints is solved based on the link and angle functions which are chosen through the graph traversal.

2.3.2 Derivation for NR-solver

For single variable optimization, to find local minimum, Newton's method is applied to the derivative f' of a twice-differentiable function f to find the roots of the derivative (solutions to $f'(x) = 0$), also known as the stationary points of f . At each iteration, $f(x)$ is approximated by a quadratic function starting from the x value in the previous iteration. The iterative scheme with step size γ for the k^{th} iteration is

$$x_{k+1} = x_k - \gamma \frac{f'(x_k)}{f''(x_k)}, \quad k \geq 0, \quad 0 \leq \gamma \leq 1 \quad (2.13)$$

In multi-dimensional problems, where there are more than one variables to be optimized, the derivative is replaced with a gradient, $\nabla F(\mathbf{x})$, and the reciprocal of the second derivative with the inverse of the Hessian matrix, $H_{F(\mathbf{x})}$. The new iterative scheme with step size γ , for the $(k+1)^{th}$ iteration, is

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \gamma [\mathbf{H}_{F(\mathbf{x}_k)}]^{-1} \nabla F(\mathbf{x}_k), \quad k \geq 0, \quad 0 \leq \gamma \leq 1 \quad (2.14)$$

The objective function $F(\mathbf{x}_k)$ as defined in Equation 2.11, its gradient vector $\nabla F(\mathbf{x}_k)$ and its Hessian matrix $[\mathbf{H}_{F(\mathbf{x}_k)}]$, for a given vector $\mathbf{x}$ in k^{th} iteration are given as, respectively,

$$F(\mathbf{x}_k) = \sum_{i=0}^l f_i^2(\mathbf{x}_k) + \sum_{j=0}^a g_j^2(\mathbf{x}_k) \quad (2.15)$$

$$\nabla F(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial F(\mathbf{x}_k)}{\partial x_p} \\ \frac{\partial F(\mathbf{x}_k)}{\partial y_p} \\ \frac{\partial F(\mathbf{x}_k)}{\partial x_q} \\ \frac{\partial F(\mathbf{x}_k)}{\partial y_q} \\ \vdots \\ \frac{\partial F(\mathbf{x}_k)}{\partial x_r} \\ \frac{\partial F(\mathbf{x}_k)}{\partial y_r} \end{bmatrix}_{2n \times 1} \quad (2.16)$$

$$\mathbf{H}_{F(\mathbf{x}_k)} = \begin{bmatrix} \frac{\partial^2 F(\mathbf{x}_k)}{\partial x_p^2} & \frac{\partial^2 F(\mathbf{x}_k)}{\partial x_p \partial y_p} & \cdots & \frac{\partial^2 F(\mathbf{x}_k)}{\partial x_p \partial y_r} \\ \frac{\partial^2 F(\mathbf{x}_k)}{\partial y_p \partial x_p} & \frac{\partial^2 F(\mathbf{x}_k)}{\partial y_p^2} & \cdots & \frac{\partial^2 F(\mathbf{x}_k)}{\partial y_p \partial y_r} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 F(\mathbf{x}_k)}{\partial y_r \partial x_p} & \frac{\partial^2 F(\mathbf{x}_k)}{\partial y_r \partial y_p} & \cdots & \frac{\partial^2 F(\mathbf{x}_k)}{\partial y_r^2} \end{bmatrix}_{2n \times 2n} \quad (2.17)$$

Each entry in $\nabla F(\mathbf{x}_k)$ and $\mathbf{H}_{F(\mathbf{x}_k)}$ is computed through the accumulation of values from all link functions and angle functions, i.e., if p and q are the unknown variables appeared in the function, then,

$$\begin{cases} \frac{\partial F(\mathbf{x}_k)}{\partial p} = \sum_{i=0}^l \frac{\partial f_i^2(\mathbf{x}_k)}{\partial p} + \sum_{j=0}^a \frac{\partial g_j^2(\mathbf{x}_k)}{\partial p} \\ \frac{\partial^2 F(\mathbf{x}_k)}{\partial p^2} = \sum_{i=0}^l \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial p^2} + \sum_{j=0}^a \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial p^2} \\ \frac{\partial^2 F(\mathbf{x}_k)}{\partial p \partial q} = \sum_{i=0}^l \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial p \partial q} + \sum_{j=0}^a \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial p \partial q} \end{cases} \quad (2.18)$$

A distance constraint function f_i can have up to four unknown variables as defined in Equation 2.4. Therefore, the partial derivatives of f_i^2 with respect to only these four variables will have to be calculated, and the rest will be zero. Similarly, there are up to sixteen entries (4×4) that have to be calculated for the Hessian matrix. In MotionGen, the calculations are done in several steps. First, calculate the common elements. Then, calculate the have-to-compute parts of the gradient vector and the Hessian matrix respectively. The common elements are

$$\left\{ \begin{array}{lcl} \mathbf{v}_i & = & [v_1, v_2]^T = [x_p - x_q, y_p - y_q]^T \\ s_i & = & \sqrt{\mathbf{v}_i^T \mathbf{v}_i} \\ e_i & = & 1 - \frac{l_i}{s_i} \\ t_1 & = & \frac{2l_i}{s_i^3} \\ t_2 & = & v_1^2 \\ t_3 & = & v_2^2 \\ t_4 & = & v_1 v_2 \\ H_1 & = & t_1 t_2 + 2e_i \\ H_2 & = & t_1 t_3 + 2e_i \\ H_3 & = & t_1 t_4 \end{array} \right. \quad (2.19)$$

With these elements, the have-to-compute part for the gradient vector is

$$\begin{bmatrix} \frac{\partial f_i^2(\mathbf{x}_k)}{\partial x_p} \\ \frac{\partial f_i^2(\mathbf{x}_k)}{\partial y_p} \\ \frac{\partial f_i^2(\mathbf{x}_k)}{\partial x_q} \\ \frac{\partial f_i^2(\mathbf{x}_k)}{\partial y_q} \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \\ -v_1 \\ -v_2 \end{bmatrix} \cdot 2e_i \quad (2.20)$$

Then, the have-to-compute part for the Hessian matrix is

$$\begin{bmatrix} \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial x_p^2} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial x_p \partial y_p} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial x_p \partial x_q} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial x_p \partial y_q} \\ \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial y_p \partial x_p} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial y_p^2} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial y_p \partial x_q} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial y_p \partial y_q} \\ \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial x_q \partial x_p} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial x_q \partial y_p} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial x_q^2} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial x_q \partial y_q} \\ \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial y_q \partial x_p} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial y_q \partial y_p} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial y_q \partial x_q} & \frac{\partial^2 f_i^2(\mathbf{x}_k)}{\partial y_q^2} \end{bmatrix} = \begin{bmatrix} H_1 & H_3 & -H_1 & -H_3 \\ H_3 & H_2 & -H_3 & -H_2 \\ -H_1 & -H_3 & H_1 & H_3 \\ -H_3 & -H_2 & H_3 & H_2 \end{bmatrix} \quad (2.21)$$

Similarly, for an angle constraint function g_j , there are up to six unknown variables, as defined in Equation 2.5 so there are six have-to-compute entries for gradient and thirty-six (6×6) for Hessian. Using the same method to derive the result, the common elements are

$$\begin{cases}
g_j = (x_p - x_q) \cdot (y_r - y_q) - (x_r - x_q) \cdot (y_p - y_q) - l_{p,q} \cdot l_{q,r} \cdot \sin(\alpha_{p,q,r}) \\
\mathbf{v}_j = [-y_q + y_r, x_q - x_r, y_p - y_r, -x_p + x_r, -y_p + y_q, x_p - x_q]^T \\
A = \begin{bmatrix} 0 & 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 \\ -1 & 0 & 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 \end{bmatrix} \cdot 2g_j \\
B = \mathbf{v}_j \mathbf{v}_j^T
\end{cases} \quad (2.22)$$

The have-to-compute part for the gradient vector is

$$\begin{bmatrix} \frac{\partial g_j^2(\mathbf{x}_k)}{\partial x_p} \\ \frac{\partial g_j^2(\mathbf{x}_k)}{\partial y_p} \\ \frac{\partial g_j^2(\mathbf{x}_k)}{\partial x_q} \\ \frac{\partial g_j^2(\mathbf{x}_k)}{\partial y_q} \\ \frac{\partial g_j^2(\mathbf{x}_k)}{\partial x_r} \\ \frac{\partial g_j^2(\mathbf{x}_k)}{\partial y_r} \end{bmatrix} = \mathbf{v}_j \cdot 2g_j \quad (2.23)$$

The have-to-compute part for the Hessian matrix is

$$\begin{bmatrix} \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_p^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_p \partial y_p} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_p \partial x_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_p \partial y_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_p \partial x_r} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_p \partial y_r} \\ \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_p^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_p \partial x_p} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_p \partial x_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_p \partial y_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_p \partial x_r} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_p \partial y_r} \\ \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_q^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_q \partial y_p} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_q^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_q \partial y_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_q \partial x_r} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_q \partial y_r} \\ \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_q^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_q \partial x_p} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_q \partial x_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_q^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_q \partial x_r} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_q \partial y_r} \\ \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_r^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_r \partial y_p} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_r \partial x_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_r \partial y_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_r^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial x_r \partial y_r} \\ \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_r^2} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_r \partial x_p} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_r \partial x_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_r \partial y_q} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_r \partial x_r} & \frac{\partial^2 g_j^2(\mathbf{x}_k)}{\partial y_r^2} \end{bmatrix} = A + B \quad (2.24)$$

Finally, part of $\mathbf{x}_k$ are the variables to be optimized because the other part of $\mathbf{x}_k$ corresponds to the solved joints. Therefore, the rows and columns

in the gradient and in the Hessian that correspond to known variables can be removed in the process. For example, for Figure 2.3, there are 7 numbers of joints, or 14 variables that are in the system of equations, but only eight of the variables need to be updated, which are $x_1, y_1, x_3, y_3, x_5, y_5, x_6, y_6$. Therefore, the size of the gradient is 8×1 , and the Hessian matrix is 8×8 .

2.3.3 NR-solver Algorithm and its Analysis

Algorithm 1 is the pseudocode that summarizes the process for the NR-solver.

Data: Initial Joint Positions, Set of Constraints
Result: Positions of all joints for *inputs*
initialization;
calculate $F(\mathbf{x}_k)$;
while *Not converged* **do**
 calculate the gradient $\nabla F(\mathbf{x}_k)$ and the Hessian $[\mathbf{H}_{F(\mathbf{x}_k)}]$;
 calculate the vector $[\mathbf{H}_{F(\mathbf{x}_k)}]^{-1}\nabla F(\mathbf{x}_k)$;
 calculate step size γ by Golden Section Method;
 calculate $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \gamma[\mathbf{H}_{F(\mathbf{x}_k)}]^{-1}\nabla F(\mathbf{x}_k)$;
 calculate the value of function $F(\mathbf{x}_{k+1})$;
end

Algorithm 1: Newton Raphson solver

Here, the Golden Section Method is used to find the best λ for the vector $[\mathbf{H}_{F(\mathbf{x}_k)}]^{-1}\nabla F(\mathbf{x}_k)$, i.e., find the best magnitude of step in that direction. Then, the convergence criteria are summarized as follows.

Convergence criteria: Three convergence criteria have been used for the simulation in MotionGen for efficiency purposes, i.e., these three allow the function to terminate within a number of iterations.

1. The objective function is considered solved if the value of $F(\mathbf{x}_k)$ falls below a certain threshold ϵ . The MotionGen user can set this value. The default value is 10^{-8} . The overall error function is of L^2 . In other words, the maximum possible link length error is 10^{-4} .
2. The function is considered converged if the difference between two successive NR iterations is less than some value. This is used together with the first criteria to determine whether the function is solved.

3. The error function may never reach zero when the solution point is outside the circuit. Therefore, if the overall error is still bigger than 10^{-8} when the maximum number of iterations is reached, it is considered no solution.

Time complexity analysis: Although the NR-solver can solve the mechanism of whatever complexity, the apparent drawback is its time complexity. A simple time complexity analysis can tell the reason. For n number of joints to solve, if there are l numbers of binary links and a numbers of actuators in this function, then,

1. Computation for the error function F takes $O(l + a)$.
2. Computation for the gradient $\nabla F(\mathbf{x}_k)$ takes $O(n \cdot (l + a))$.
3. Computation for the Hessian $[\mathbf{H}_{F(\mathbf{x}_k)}]$ takes $O(n^2 \cdot (l + a))$. Each time, for the $(2n \times 2n)$ -sized Hessian matrix, a constraint function is to visit all entries in the matrix to check which entry to update.
4. Computation for the inverse of Hessian $\mathbf{H}^{-1}$ takes $O(n^3)$ (Naïve method)
5. Computation for matrix-vector multiplication $\mathbf{H}^{-1}\nabla F(\mathbf{x}_k)$ takes $O(n^2)$
6. These computations are done multiple times. There is a big constant $O(1)$ that multiplies with the steps above.

$(l + a)$ is always some number greater or equal to $2n$ for planar mechanics because there are $2n$ numbers of unknown variables in the system of equations and the number of functions has to be greater or equal to this value. Thus, the computation for the Hessian takes the most time, which is comparable to $O(n^3)$ or even bigger if l and a are badly chosen. For example, inside an n -ary link, there can be $\frac{n(n-1)}{2}$ numbers of binary links. A bad selection for computation may result in $O(n^4)$ for Hessian matrix calculation and is computationally expensive. To illustrate, we experimented that when n is greater or equal to 30, the time to finish the computation for a circuit will take more than one second for a modern personal computer spec.

Furthermore, the NR-solver converges slowly to the desired error range for some complex mechanisms, i.e., the loop is made multiple times.

2.3.4 System of Equations Formulation for GN-solver

There is another well-known Newton-based model to solve k variables with m equations, with $m > k$. It uses the Jacobian matrix to approximate the Hessian matrix instead. In this context, $m = l+a$ and $k = 2n$. The constraint functions are stacked in a matrix. The system of equations is

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_l \\ g_1 \\ \vdots \\ g_{a-1} \\ g_a \end{bmatrix}_{(l+a) \times 1} \quad (2.25)$$

For this $\mathbf{f}$ vector, the overall cost function is written as

$$F(\mathbf{x}) = \mathbf{f}(\mathbf{x})^T \mathbf{f}(\mathbf{x}) \quad (2.26)$$

This is another way to write the sum of squared error compared to Equation 2.15. Then, the corresponding Jacobian matrix is

$$\mathbf{J} = \begin{bmatrix} \frac{\delta f_1}{\delta x_p} & \frac{\delta f_1}{\delta y_p} & \cdots & \frac{\delta f_1}{\delta x_r} & \frac{\delta f_1}{\delta y_r} \\ \frac{\delta f_2}{\delta x_p} & \frac{\delta f_2}{\delta y_p} & \cdots & \frac{\delta f_2}{\delta x_r} & \frac{\delta f_2}{\delta y_r} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \frac{\delta g_{a-1}}{\delta x_p} & \frac{\delta g_{a-1}}{\delta y_p} & \cdots & \frac{\delta g_{a-1}}{\delta x_r} & \frac{\delta g_{a-1}}{\delta y_r} \\ \frac{\delta g_a}{\delta x_p} & \frac{\delta g_a}{\delta y_p} & \cdots & \frac{\delta g_a}{\delta x_r} & \frac{\delta g_a}{\delta y_r} \end{bmatrix}_{(l+a) \times (2n)} \quad (2.27)$$

The derivation for these constraint functions is very similar to the NR-solver. The derivations for a distance constraint f_i is

$$\begin{bmatrix} \frac{\partial f_i(\mathbf{x})}{\partial x_p} \\ \frac{\partial f_i(\mathbf{x})}{\partial y_p} \\ \frac{\partial f_i(\mathbf{x})}{\partial x_q} \\ \frac{\partial f_i(\mathbf{x})}{\partial y_q} \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \\ -v_1 \\ -v_2 \end{bmatrix} \cdot \frac{1}{s_i} \quad (2.28)$$

v_1, v_2, s_i are the common elements shown in Equation 2.19. Then, the derivations for a angle constraint g_j is

$$\begin{bmatrix} \frac{\partial g_j(\mathbf{x})}{\partial x_p} \\ \frac{\partial g_j(\mathbf{x})}{\partial y_p} \\ \frac{\partial g_j(\mathbf{x})}{\partial x_q} \\ \frac{\partial g_j(\mathbf{x})}{\partial y_q} \\ \frac{\partial g_j(\mathbf{x})}{\partial x_r} \\ \frac{\partial g_j(\mathbf{x})}{\partial y_r} \end{bmatrix} = \mathbf{v}_j \quad (2.29)$$

$\mathbf{v}_j$ is the common element shown in Equation 2.22. Finally, the update policy becomes

$$\mathbf{x} \leftarrow \mathbf{x} - \gamma(\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \mathbf{f} \quad (2.30)$$

The GN-solver is intriguing because it does not need to compute for a Hessian matrix and computing a Jacobian matrix is almost the same as computing the gradient of the NR model that is discussed in the previous sections, resulting in a simpler value stacking process. However, the symmetric matrix $\mathbf{J}^T \mathbf{J}$ can sometimes have a determinant of zero, which becomes an uninvertible matrix, meaning this update policy is not robust. To address this problem, we added a “damping factor” into the matrix in Equation 2.31.

$$\mathbf{x} \leftarrow \mathbf{x} - \gamma(\mathbf{J}^T \mathbf{J} + \lambda \cdot \text{diag}(\mathbf{J}^T \mathbf{J}))^{-1} \mathbf{J}^T \mathbf{f} \quad (2.31)$$

We set $\lambda = 1$ and diag stands for the diagonal matrix composed by the diagonal values of the input matrix. γ is determined through the golden section method. The pseudocode for GN-solver is shown in Algorithm 2, which is slightly modified compared to Algorithm 1.

The damping factor idea comes from the Levenberg-Marquardt algorithm (LMA) [25]. The difference between LMA and our update policy is that we fix the value of λ and compute γ using the golden section, while LMA computes λ to find the best step size for each iteration. The convergence criterion is the same as in Section 2.3.3.

The GN-solver’s time complexity is the same as the NR-solver, which is $O(n^2 \cdot (l + a))$, contributed by the $\mathbf{J}^T \mathbf{J}$ matrix multiplication. However, for most mechanisms, the GN-solver tends to converge faster than the NR-solver. In other words, it takes fewer numbers of iterations to get a good

Data: Initial Joint Positions, Set of Constraints
Result: Positions of all joints for *inputs*
initialization;
calculate $F(\mathbf{x}_k)$;
while *Not converged* **do**
 calculate the Jacobian $\mathbf{J}$;
 calculate $\mathbf{J}^T \mathbf{J}$ and $\det(\mathbf{J}^T \mathbf{J})$;
 if $\det(\mathbf{J}^T \mathbf{J}) = 0$ then $\lambda \leftarrow 1$; else, $\lambda \leftarrow 0$;
 calculate $(\mathbf{J}^T \mathbf{J} + \lambda \cdot \text{diag}(\mathbf{J}^T \mathbf{J}))^{-1} \mathbf{J}^T \mathbf{f}$;
 calculate step size γ by Golden Section Method;
 calculate $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \gamma(\mathbf{J}^T \mathbf{J} + \lambda \cdot \text{diag}(\mathbf{J}^T \mathbf{J}))^{-1} \mathbf{J}^T \mathbf{f}$;
 calculate the value of function $F(\mathbf{x}_{k+1})$;
end

Algorithm 2: Gauss Newton solver

result. Both solvers are provided in the current MotionGen and the default optimization-based solver is the GN-solver.

2.4 Removal of the Forward Chain Solver

We removed the forward chain solver in this iteration. In brief, it can compute the joints actuated by the rotary actuator. This solver is commonly used for open-loop robot arms and can only compute all the joint positions in the actuator-actuator chain. Furthermore, the chain has to start from the ground, so this solver has two critical limitations.

Because the angle constraint is introduced to the ArcSect solver and the optimization-based solver, they can handle the actuator chain. Therefore, the forward chain solver becomes unnecessary. Additionally, both the ArcSect solver and the Newton solver can handle actuators regardless of whether or not they are in the actuator-actuator chain. Furthermore, although the time complexity for the forward chain solver is $O(1)$ for one joint, i.e., $O(n)$ for n joints and it has slightly fewer computations per joint than the ArcSect solver, their time difference is minimal. As a result, the forward chain solver gets removed in this iteration.

2.5 Conclusion

A comprehensive comparison of all the mentioned solvers is demonstrated in Table 2.1. In summary, this chapter first introduces two types of planar kinematic constraint equations. After that, two major types of solvers are presented, where one of them is removed, and three solvers are implemented in the latest version. These solvers will be used after the extraction of the solving procedure.

Solver Name	Type	Time Cost	Pros and Cons
Forward Chain (Removed)	Grapho- Analytical	$O(n)$	Fastest but has very limited scope.
ArcSect	Grapho- Analytical	$O(n)$	Fast but cannot handle complex structures.
NR-solver	Numerical Optimization	$O(n^2(a + l))$	Can solve all structures, always converges, but it is slow and time cost is a polynomial increase.
GN-solver	Numerical Optimization	$O(n^2(a + l))$	Can solve all structures, always converges, sometimes faster than NR-solver, but it is slow and its time cost is a polynomial increase.

Table 2.1: Solver comparison table

Chapter 3

Arrangement of Mechanical Components

The software implementation of simulation code deeply connects with the arrangement of data. During the development of MotionGen, we develop a set of representations to help ourselves identify the problems and help ourselves organize the data pre-simulation. This chapter, together with Chapter 2, serves as the preparation for the extraction of the solving procedure in Chapter 4.

This chapter is split into five sections. Section 3.1 reviews the basic components of a planar linkage mechanism. Section 3.2 introduces the compact data format to save all the necessary information. Section 3.3 is about converting the prismatic joints into a revolute joint. Then, Section 3.4 will go through the operations on the static components. Finally, the dynamic components are discussed in Section 3.5.

3.1 Basic Components of a Linkage Mechanism

Four types of components together define a planar linkage mechanism and are saved in a nested object in MotionGen. Each object is given a unique string for identification. All linkage mechanisms in MotionGen can be described with the components below.

1. **joints**: These are the points of interest in the mechanism. Unless

specified, a revolute joint is referred to as a joint in this thesis. In planar mechanics, a revolute(R) joint is a point in the plane. Additionally, a prismatic(P) joint can be reinterpreted into a combination of links and revolute joints.

2. **links:** The particular collection of joints defines a link. The collection of joints is in the same rigid body. The simplest link is a binary link (two joints) and there can be an n -ary link, where n is the number of joints in this link. All joints on the link are revolute(R) joints for simulation. Furthermore, by the rigid body's definition, every two joints in the n -ary link collection correspond to a binary link.
3. **slots:** A slot is a finite straight slider and is the interpretation of a prismatic(P) joint. A slot is defined by two joints, which are at the two ends of the slot respectively. Additionally, the distance between the two joints is fixed because these two joints are on the same link. Jointly together, the two joints define the slot axis. If a joint is in a slot, then this joint must stay on the slot axis throughout its entire motion.
4. **actuators:** There are two types of actuators, rotary and linear. A rotary actuator contributes one angle constraint, which needs at least three joints to define. A linear actuator is seen in a slot, where two end joints define the moving direction and the moving joint is some distance away from one of the two end joints.

In particular, joints, links, slots, and the links and joints introduced through the reinterpretation of slots are the *static components* because their properties (e.g., link length, connection with other components) are unchanged throughout the process. On the other hand, the actuators and the dynamic links introduced through the interpretation of actuators are the *dynamic components* because their properties can change during the process.

3.2 Representation of a Planar Linkage using Matrices

The previous code implementation saved all information in strings, resulting in poor data tractability for programming. To address this, we introduced

graph theory notations into the current iteration. Lung-Wen Tsai has already done extensive research in representing kinematic structures [26]. He used the adjacency matrix (denoted with A) and incidence matrix (denoted with B . Some call it LJ-matrix [27]) to indicate the structure of a mechanism. The adjacency matrix A denotes whether or not two links are connected through a joint. Then, the incidence matrix B denotes which joint is on which link.

Accordingly, the adjacency matrix A is defined as follows:

$$A_{i,j} = \begin{cases} 1 & \text{if link } i \text{ is connected to link } j \text{ by a joint,} \\ 0 & \text{otherwise (including } i = j). \end{cases} \quad (3.1)$$

This matrix of link-to-link format is not used for our simulation code. However, the incidence matrix, which is of link-to-joint format, is very useful. The component in the i^{th} row and j^{th} column of the incidence matrix B is

$$B_{i,j} = \begin{cases} 1 & \text{if link } i \text{ contains joint } j, \\ 0 & \text{otherwise.} \end{cases} \quad (3.2)$$

B does not save whether or not a link is a ground link. This property can be introduced with the L column vector which is defined as,

$$L_{i,0} = \begin{cases} 1 & \text{if link } i \text{ is a ground link,} \\ 0 & \text{if link } i \text{ is a moving link.} \end{cases} \quad (3.3)$$

B and L can denote a linkage mechanism together. For example, Figure 3.1 is a common four-bar linkage mechanism, which accordingly can be denoted as shown in Equation 3.4.

$$[L|B] = \begin{matrix} & L & j_0 & j_1 & j_2 & j_3 & j_4 \\ \begin{matrix} l_0 \\ l_1 \\ l_2 \\ l_3 \end{matrix} & \left(\begin{array}{c|ccccc} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{array} \right) \end{matrix} \quad (3.4)$$

$4 \times (1+5)$

There are other useful matrices to denote planar linkage mechanisms as well. We used a joint-to-joint form matrix T because it can store all binary links. The matrix T is defined as,

$$T_{i,j} = \begin{cases} 1 & \text{if joint } i \text{ is connected to joint } j, \text{ excluding } i = j, \\ 0 & \text{otherwise.} \end{cases} \quad (3.5)$$

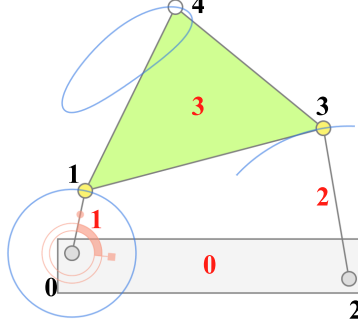


Figure 3.1: This is a common four-bar mechanism. The joints are marked with numbers in black and the links are marked with numbers in red.

The size of a T matrix is $n \times n$, where n is the number of joints. This T matrix is introduced to save graph traversal results. In the graph traversal stage, whether a binary link exists or not is the only important thing and the real distances are not as important. Moreover, this joint-to-joint format can be utilized to save the distance between two points, in other words, the binary link length. In fact, T is a shallow version of the *distance matrix* D , which collects all distance constraints and is defined as,

$$D_{i,j} = \begin{cases} l_{i,j} & \text{excluding } i = j, \\ 0 & i = j \end{cases} \quad (3.6)$$

Here, $l_{i,j}$ is the distance between joint i and j , which is unchanged during the entire simulation process. Additionally, $l_{i,j}$ is equal to $l_{j,i}$ because a binary link is undirected. Some new binary links can be found during the process, so D can be computed using initial positions and the updated T . This update policy is

$$D_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \cdot T_{i,j} \quad (3.7)$$

Similar to links, for joints, the notation of ground joint can be saved in a row vector J which is defined as,

$$J_{0,i} = \begin{cases} 1 & \text{if joint } i \text{ is a ground joint,} \\ 0 & \text{otherwise.} \end{cases} \quad (3.8)$$

Additionally, because the size of J is $1 \times n$ (n is the number of joints), it can also be stored in the diagonal of T . For example, with such definition, the four-bar mechanism in Figure 3.1 can be represented as shown in Equation 3.9. Here, the *diag* function creates a matrix whose diagonal elements are from J and the non-diagonal elements are zero.

$$\text{diag}(J) + T = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}_{5 \times 5} \quad (3.9)$$

Next, because three joints define an actuator, if we follow the same joint-to-joint pattern as D and T to denote the actuators, then the data structure would be three-dimensional (joint-to-joint-to-joint). In other words, the actuators will be saved in a three-dimensional tensor. This wastes lots of space unnecessarily. Instead, we save the actuators in *objects and pointers format* and store them all together in one matrix because the notations of the actuators share the same size. For example, in Figure 3.1, the actuator corresponds to the angle constraint between $\vec{v}_{0,2}$ (the ground binary link) and $\vec{v}_{0,1}$ (the moving binary link) is $(2, 0, 1, \text{"R"})$, where the order matters.

Thus, a general rotary actuator $\alpha_{i,j,k}$ can be represented as $(i, j, k, \text{"R"})$, where the angle constraint is between $\vec{v}_{j,i}$ and $\vec{v}_{j,k}$. Similarly, a general linear actuator would be $(i, j, k, \text{"P"})$, because they exist in the slot. However, different from a rotary actuator, j_i and j_k are the two end joints of the slot, and j_k is the moving joint in the slot. Hence $\vec{v}_{i,k}$ denotes the positive direction, and $\vec{v}_{i,j}$ corresponds to the dynamic distance between the and the slot joint. In such notation, the actuators can be stacked together in the *input matrix* I , which can be, for example,

$$I = \begin{bmatrix} i & j & k & R \\ \vdots & \vdots & \vdots & \vdots \\ p & q & r & P \end{bmatrix}_{a \times 4} \quad (3.10)$$

This format can be applied to slots as well. Therefore, to save all these relationships of three joints (two end joints and one slot joint), the *slot matrix* S is introduced. An example is in Equation 3.11, where m is the number of joints in the slots of the mechanism.

$$S = \begin{bmatrix} i & j & k \\ \vdots & \vdots & \vdots \\ p & q & r \end{bmatrix}_{m \times 3} \quad (3.11)$$

Up till now, the aforementioned matrices L , J , B , D , T , I , and S do not save the joint positions. Hence, another object of saving these joint positions is necessary. Therefore, we can stack the joints as shown in Equation 3.12.

$$P_k = \begin{matrix} & \begin{matrix} x & y \end{matrix} \\ \begin{matrix} j_0 \\ j_1 \\ j_2 \\ \vdots \\ j_{n-1} \end{matrix} & \begin{pmatrix} x_0 & y_0 \\ x_1 & y_1 \\ x_2 & y_2 \\ \vdots & \vdots \\ x_n & y_n \end{pmatrix} \end{matrix}_{n \times 2} \quad (3.12)$$

One specific state is determined by a set of inputs. The notation k in Equation 3.12 means this is the k^{th} state for a mechanism and we can denote P_{init} for the initial state. If there are s number of states in one simulation, they can be stacked together to form a *position tensor* P :

$$P = [P_0, P_1, P_2, \dots, P_{s-1}]_{s \times n \times 2} \quad (3.13)$$

Last but not least, the combination of the input from the actuators results in one state. Therefore, corresponding to P , we can also stack the actuator inputs of each state together in the *velocity matrix* V as shown in Equation 3.14.

$$V = \begin{matrix} & \begin{matrix} s_0 & s_1 & \cdots & s_{s-1} \end{matrix} \\ \begin{pmatrix} \alpha_0(t_0) & \alpha_0(t_1) & \cdots & \alpha_0(t_{s-1}) \\ \alpha_1(t_0) & \alpha_1(t_1) & \cdots & \alpha_1(t_{s-1}) \\ \alpha_2(t_0) & \alpha_2(t_1) & \cdots & \alpha_2(t_{s-1}) \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_{a-1}(t_0) & \alpha_{a-1}(t_1) & \cdots & \alpha_{a-1}(t_{s-1}) \end{pmatrix} \end{matrix}_{a \times s} \quad (3.14)$$

Representing a planar linkage mechanism: Without using the reinterpretation tricks in Section 3.3 and Section 3.5, one can always save the mechanism in a set of matrices. Generally, for a mechanism M , if there are l

number of links, a number of actuators, n number of joints, m number of slot relationships, and s number of states that are successfully simulated, then all these can be stored in a compact format denoted in Equation 3.15. In brief, J , L , B , D , S and I save all types of constraints. V and P store the simulation results of all states for input and joint positions, respectively.

$$M = \begin{cases} J & : J_{1 \times n}, \\ L & : L_{l \times 1}, \\ B & : B_{l \times n}, \\ D & : D_{n \times n}, \\ S & : S_{m \times 3}, \\ I & : I_{a \times 4}, \\ V & : V_{a \times s}, \\ P & : P_{s \times n \times 2} \end{cases} \quad (3.15)$$

There are some redundancies in this format. However, they are proved to be helpful in graph traversal operations. Furthermore, matrices are used to store the mechanism because they only contain numbers and are easy to reinterpret. Comparatively, in Section 3.4, the adjacency list format is introduced because this format is generally more time-efficient for iterative search.

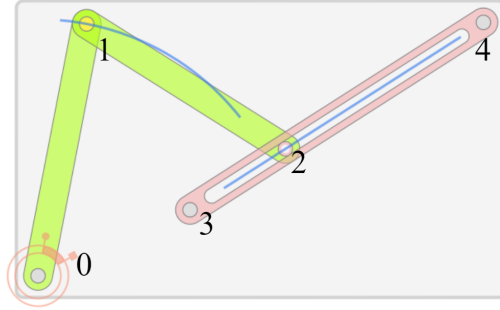


Figure 3.2: A RRRP mechanism

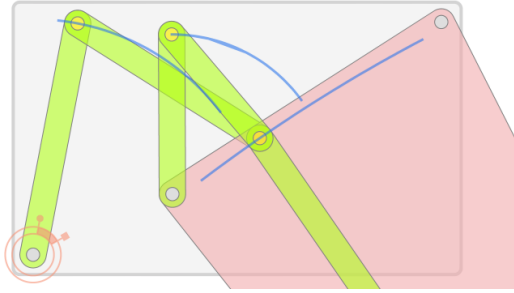


Figure 3.3: This is the reinterpreted RRRP mechanism for Figure 3.2. A joint is in the far bottom-right.

3.3 Reinterpretation of Prismatic(P) Joint

In MotionGen, the slots are used to perform functions of prismatic joints. Figure 3.2 shows an example mechanism with a slot (pink). MotionGen implements the slot concept instead of a prismatic joint that can go to infinity because those designs are not practical. Additionally, slots fit the actual design of SnappyXO-based mechanisms. Furthermore, it is more intuitive to create a slot than an(a) RR/RP/PR/PP dyad.

In Figure 3.2, the joint in the slot, or *slot joint*, is performing a straight line motion with respect to the slot. A straight line can be reinterpreted as the arc of a circle, whose radius is large enough such that the curvature is next to zero. Therefore, we can approximate a straight line with a circle instead. Then, the intersections of the straight line and the arc are on the edges of the slots. The circle's center is perpendicular to this straight line and is in the perpendicular bisector.

The exemplar reinterpretation can be seen in Figure 3.3. First, the slot is removed, with a newly added third joint on the slot link. Then, two constraining links and a constraining joint are added to constrain the moving space, pointing left-upward in this figure. The slot is removed and the joint previously in the slot is moving on an arc. When the third joint goes further, this arc tends to be flat. Therefore, the difference between the arc and the straight line can be controlled by changing the distance of the third joint from the two end joints.

Through the reinterpretation trick, the third joint, or *far joint* j_f , can be computed by the steps below:

1. Compute vector between the two edge joints of the slot. Denote it as $[v_x, v_y]$. The perpendicular vector is $[-v_y, v_x]$. The normalized vector is $\vec{v} = \frac{[-v_y, v_x]}{\sqrt{v_y^2 + v_x^2}}$.
2. This vector goes through the middle point of the line between the two edges. If the first end joint position is $[x_1, y_1]$ and the second end joint position is $[x_2, y_2]$, then the middle point is $p_{mid} = [\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2}]$.
3. Set the arc radius as height h for the triangle formed by the three joints in the slot. Then the third joint position is $p_{mid} + \vec{v} \cdot h$.

The value of h depends on the required precision. In MotionGen, this h is set dynamically such that the deviation of the point would not exceed some length. The expressions below give the deviation from the curve to the straight line. First, the distance between two edge joints is

$$d = \sqrt{v_x^2 + v_y^2} \quad (3.16)$$

The error is measured with the distance between the point on the arc and its corresponding point in the straight line, which is

$$err = \sqrt{\left(\frac{d}{2}\right)^2 + (h)^2} - h \quad (3.17)$$

The expression is equal to

$$(err + h)^2 = err^2 + h^2 + 2err \cdot h = \frac{d^2}{4} + h^2 \quad (3.18)$$

err^2 is extremely small compared to other elements. Therefore, by removing err^2 and removing h^2 on both sides, we get

$$h = \frac{d^2}{8err} \quad (3.19)$$

The next to compute is the *constraining joint*, which connects the two constraining links and is denoted as j_c . There are several steps to compute the position of this joint:

1. One of the end joints is selected as the anchor joint for constraint. Usually, this would be the first joint in the slot, which is denoted as j_p . Then the other joint is denoted as j_r .
2. Compute the length of the two constraining links. There are two offsets, one for the least distance from the slot joint j_q to j_p and the other for the least distance from j_q to j_r , which are o_p and o_r respectively. When the two links are fully stretched, j_q is o_r away from the j_r in the slot. When two links are folded, j_q is o_p away from the anchor joint (j_p). Therefore,

$$\begin{cases} l_{p,c} + l_{q,c} = d - o_r \\ l_{p,c} - l_{q,c} = o_p \end{cases} \quad (3.20)$$

And the solution is

$$\begin{cases} l_{p,c} = \frac{d+o_p-o_r}{2} \\ l_{q,c} = \frac{d-o_p-o_r}{2} \end{cases} \quad (3.21)$$

3. Compute the position of the constraining joint j_c using arc intersection. The first circle's center is j_p and the radius is $l_{p,c}$. The second circle's center is j_q and the radius is $l_{q,c}$.

For each slot relationship in the slot matrix S , the converting procedure is done repeatedly until every one of them is represented with revolute joints and binary links. Since new links and joints are added to the system, the matrices are expanded accordingly. An example will be shown in Section 3.4.

To end this section, we present nine mechanisms to show that such a formulation of mechanisms can handle all combinations of RR, RP, PR, and PP dyads in Figure 3.4.

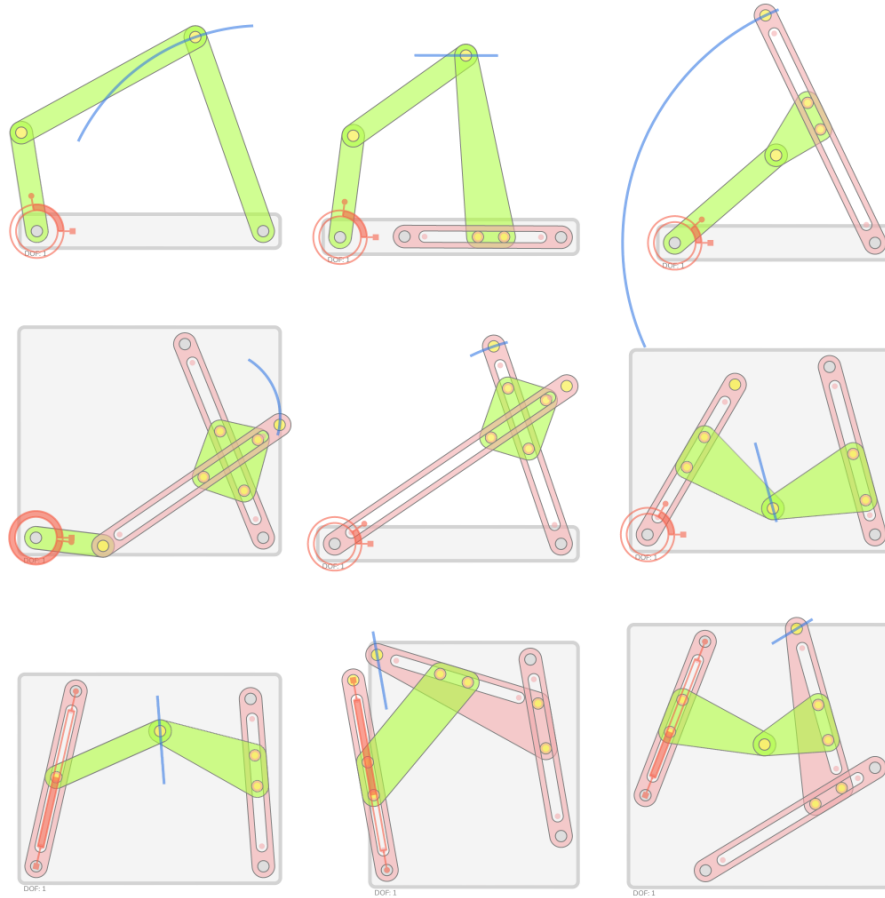


Figure 3.4: All nine combinations of RR, PR, RP, and PP dyads are displayed above. The first row shows RRRR, RRRP, and RRPR mechanisms. The second row shows RRPP, RPPR, and RPRP mechanisms. The third row shows PRRP, RPPP, and PRPP mechanisms. All nine mechanisms can be simulated.

3.4 Operations on Static Components

This section handles the static components, which have unchanged properties throughout the simulation. This step is necessary because some potentially useful static components can be found. This will be further discussed in Chapter 4.

Data information analysis: Of all these data formats, A can be computed using B , J , L , and T . D can be computed after graph traversal using T and P_{init} . S and I can be interpreted or saved separately. The tensor P is a result of V . From the kinematic structural analysis point of view, B , J , L , and T are the four necessary matrices to do the structural analysis.

Matrix setup for static components: Matrix B , J , L , and T are used to represent the process. P_{init} will be updated when new joints are added during the reinterpretation of slots. Therefore, the matrix B , J , and T are expanded accordingly. The new joints and new links are added to the new columns and rows. Additionally, J can be stored in the diagonal of T during the operation.

For example, when the mechanism in Figure 3.2 is reinterpreted, B is as shown in Equation 3.22, where j_5 is the far joint and j_6 is the constraining joint, l_4 is the link between the slot joint j_2 and the far joint j_5 . The constraining joint j_6 connects the two constraining links l_5 and l_6 . Initially, the slots and the dynamic link are not interpreted as connections. The third joint for the slot is added first, then the constraining joint is added later.

$$\begin{matrix} l_0 \\ l_1 \\ l_2 \\ l_3 \end{matrix} \begin{pmatrix} j_0 & j_1 & j_2 & j_3 & j_4 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix} \rightarrow \begin{matrix} l_0 \\ l_1 \\ l_2 \\ l_3 \\ l_4 \\ l_5 \\ l_6 \end{matrix} \left(\begin{array}{ccccc|cc} j_0 & j_1 & j_2 & j_3 & j_4 & j_5 & j_6 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ \hline 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{array} \right) \quad (3.22)$$

Then, for matrices T and J , the change is shown in Equation 3.23.

$$\begin{matrix} & \begin{matrix} j_0 & j_1 & j_2 & j_3 & j_4 \end{matrix} \\ \begin{matrix} j_0 \\ j_1 \\ j_2 \\ j_3 \\ j_4 \end{matrix} & \begin{pmatrix} 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 \end{pmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} j_0 & j_1 & j_2 & j_3 & j_4 & j_5 & j_6 \end{matrix} \\ \begin{matrix} j_0 \\ j_1 \\ j_2 \\ j_3 \\ j_4 \\ j_5 \\ j_6 \end{matrix} & \left(\begin{array}{ccccc|cc} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ \hline 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \end{array} \right) \end{matrix} \quad (3.23)$$

More generally, Algorithm 3, Algorithm 4 and Algorithm 5 summarize the processes of building up the T matrix (J is stored in its diagonal), B and P_{init} matrices respectively.

Adjacency list format: For example, the mechanism in Figure 3.3 can be represented in link-major order and joint-major order shown in Table 3.1 and Table 3.2 for its B matrix respectively. Both are representing the mechanism in the *adjacency list format* and store the same set of information in B .

Comparatively, the matrix format is used by J , L , B , D , T , which is good at looking for a specific property, e.g., the binary distance between j_i and j_j is stored in $D_{i,j}$ and in $D_{j,i}$. However, iterating through the elements in these matrices is less efficient than using the adjacency list because there are fewer components in the list. Furthermore, the iteration is mainly done for the information stored in B . Therefore, Chapter 4 will use the adjacency list format for matrix B . One particular operation is the *link merge operation*. This will be explained after Section 3.5.

Link Merge: In the process of the extraction of the solving procedure, it is possible that two or more links can be merged into one. The adjacency list format for Figure 3.3 shows that, in the intermediate state of graph analysis, j_3 and j_4 are ground joints while their link is movable (not merged with the ground link). Inside the simulation code, this conflicting property and alike will be updated during the solution arrangement stage. For this particular scenario, l_4 will be merged with the ground link l_0 at some stage.

Another simple example is in Figure 3.5, where two ternary links share two joints, which uses the same logic as the previous example. Therefore, these two links will be merged into one 4-ary link. Similar scenarios can

Link ID (entry)	Has Joints (Joint ID)
0	0, 3, 4
1	0, 1
2	1, 2
3	3, 4, 5
4	2, 5
5	2, 6
6	3, 6

Table 3.1: This is the adjacency list format of storage for Figure 3.3 in link major order. Compared to the B matrix in Equation 3.23, the exclusion of joints for a particular link is expressed through not saving it in the list.

Joint ID (entry)	Has Links (Link ID)
0	0, 1
1	1, 2
2	2, 4, 5
3	0, 3, 6
4	0, 3
5	3, 4
6	5, 6

Table 3.2: This is the adjacency format in joint major order for Figure 3.3.

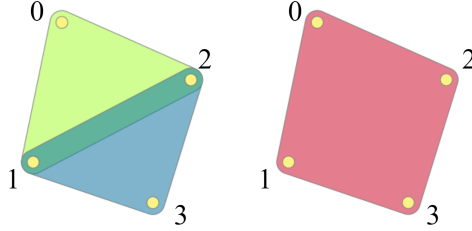


Figure 3.5: The two ternary links (green and blue) share two joints, meaning these two links are rigidly connected and can be seen as a 4-ary link (red).

happen to other combinations of joints when the user draws the structure chaotically. Of course, there are other conditions that a particular set of links can be merged into one link with more joints, but they will be discussed extensively in Chapter 4.

Without going into the details of the conditions to merge the links, this section only introduces the link merge operation by creating a new link that collects all the joints in the selected links. This is a union operation for joint sets and is done with the adjacency list B_l , which is the link major order representation of B . Then, the resultant new binary link can be mapped to T accordingly. Furthermore, if a ground link is inside the set of links to merge, the new link is a ground link. Else, it is a moving link. Finally, the joint property is mapped to J (diagonal of T) accordingly.

As an example, for Figure 3.5, the change of B_l is shown in Equation 3.24, where l_g , l_b , l_r are the green, blue and red link respectively.

$$\begin{array}{|c|c|} \hline \text{Link ID} & \text{Has Joints} \\ \hline \text{(entry)} & \text{(Joint ID)} \\ \hline g & 0, 1, 2 \\ \hline b & 1, 2, 3 \\ \hline \end{array} \rightarrow \begin{array}{|c|c|} \hline \text{Link ID} & \text{Has Joints} \\ \hline \text{(entry)} & \text{(Joint ID)} \\ \hline r & 0, 1, 2, 3 \\ \hline \end{array} \quad (3.24)$$

The change of T is shown in Equation 3.25. Apparently, a new binary link between j_0 and j_3 is added.

$$\begin{array}{c} j_0 \\ j_1 \\ j_2 \\ j_3 \end{array} \begin{pmatrix} j_0 & j_1 & j_2 & j_3 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix} \rightarrow \begin{array}{c} j_0 \\ j_1 \\ j_2 \\ j_3 \end{array} \begin{pmatrix} j_0 & j_1 & j_2 & j_3 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \quad (3.25)$$

This is written in this order because, during a merge operation, B_l is computed before T . After all, all binary links exist in the n -ary links. Furthermore, T is not converted to the list format because we want to utilize the advantage of the matrix format in searching for a specific binary link. Algorithm 6 summarizes the link merge operation.

After a slot reinterpretation and a proper merge operation, only the dynamic components remained not discussed. Therefore, the following section will describe how the actuators are handled in the simulator.

Data: $joints, links, slots$

Result: B matrix

Sort joints into order; Sort links into order (Ground link to be 0, others to be 1, 2, ..., $l - 1$);

$m \leftarrow \text{len}(links); n \leftarrow \text{len}(joints);$

$B \leftarrow \mathbf{0}_{m \times n};$

foreach $l_y \in links$ **do**

foreach $j_x \in \text{JointOf}(l_y)$ **do**

$B[y, x] \leftarrow 1;$

end

end

foreach $slot_{p,r} \in slots$ **do**

$l_y \leftarrow \text{linkOf}([p, r]);$

 Expand $B_{m \times n}$ to $B_{m \times (n+1)}$ with zeros on right and bottom edge;

$f \leftarrow n;$

$B[y, f] \leftarrow 1;$

$n \leftarrow n + 1;$

foreach $j_q \in \text{jointsOn}(slot_{p,r})$ **do**

 Expand $B_{m \times n}$ to $B_{(m+3) \times (n+1)}$ with zeros on right and bottom edge;

$c \leftarrow n;$

 Assign 1 to $(m, f), (m, q), (m + 1, q), (m + 1, c), (m + 2, p), (m + 2, c)$ entries of B ;

$m \leftarrow m + 3;$

$n \leftarrow n + 1;$

end

end

Algorithm 3: Algorithm for B matrix initialization pre-simulation

Data: $joints, links, slots$

Result: T matrix (J is in its diagonal)

Sort joints into order;

$n \leftarrow \text{len}(joints)$;

$T \leftarrow \mathbf{0}_{n \times n}$;

foreach $p \leftarrow 0$ to $\text{len}(joints)$ **do**

if j_p is ground joint **then**

$T_{p,p} \leftarrow 1$;

end

end

foreach $link \in links$ **do**

foreach $j_p \in \text{JointOf}(link)$ **do**

foreach $j_q \in \text{JointOf}(link)$ **do**

if $p \neq q$ **then**

$T_{p,q} \leftarrow 1$;

end

end

end

end

foreach $slot_{p,r} \in slots$ **do**

 Expand $T_{n \times n}$ to $T_{(n+1) \times (n+1)}$ with zeros on right and bottom edge;

$f \leftarrow n$;

 Assign 1 to $(p, f), (f, p), (r, f), (f, r)$ entries of T ;

$n \leftarrow n + 1$;

foreach $j_q \in \text{jointsOn}(slot_{p,r})$ **do**

 Expand $T_{n \times n}$ to $T_{(n+1) \times (n+1)}$ with zeros on right and bottom edge;

$c \leftarrow n$;

 Assign 1 to $(p, c), (c, p), (q, c), (c, q), (q, f), (f, q)$ entries of T ;

$n \leftarrow n + 1$;

end

end

Algorithm 4: Algorithm for T matrix initialization pre-simulation. J is in the diagonal of T .

Data: $joints, links, slots, err$ and offsets
Result: P matrix
Sort joints into order;
 $n \leftarrow \text{len}(joints)$;
 $P \leftarrow \mathbf{0}_{n \times 2}$;
foreach $j_i \in joints$ **do**
| $P[i] \leftarrow j_i$;
end
foreach $slot_{p,r} \in slots$ **do**
| Expand $P_{n \times 2}$ to $P_{(n+1) \times 2}$ with zeros on bottom edge;
| Compute the j_f given j_p, j_r, err ;
| $f \leftarrow n$;
| $P[f] \leftarrow j_f$;
| $n \leftarrow n + 1$;
| **foreach** $j_q \in \text{jointsOn}(slot_{p,r})$ **do**
| | Expand $P_{n \times 2}$ to $P_{(n+1) \times 2}$ with zeros on bottom edge;
| | $c \leftarrow n$;
| | Compute the $l_{p,c}, l_{q,c}$ given j_p, j_r , offsets from p, r slot;
| | Compute j_c given $l_{p,c}, l_{q,c}, j_p, j_q$;
| | $P[c] \leftarrow j_c$;
| | $n \leftarrow n + 1$;
| **end**
end

Algorithm 5: Algorithm for P matrix initialization pre-simulation

Data: B_l (link-major format of B), L , T (J in its diagonal), linkID
Result: Updated B_l , L , T (J in its diagonal)
 Remove links in L whose IDs are in linkIDs; (Size reduce)
 If 0 is in linkIDs, add element to L from its head; else, push element to L at its end; (Size expand)
 If 0 is in linkIDs, $n \leftarrow 0$; else, $n \leftarrow \text{len}(J)$;
 Create an empty link object l_{new} ;
foreach $l_i \in B_l$ **do**
 if $i \in \text{linkIDs}$ **then**
 $l_{new} \leftarrow \text{Union}(\text{JointOf}(l_{new}), \text{JointOf}(l_i))$;
 end
end
 Remove links in B_l whose IDs are in linkIDs;
 Assign l_{new} to n entry in B_l ;
foreach $j_p \in \text{JointOf}(B_{new})$ **do**
 foreach $j_q \in \text{JointOf}(B_{new})$ **do**
 if $p == q$ and 0 is in linkIDs **then**
 $T_{p,q} \leftarrow 1$;
 end
 if $p \neq q$ **then**
 $T_{p,q} \leftarrow 1$;
 end
 end
end
end

Algorithm 6: Algorithm for link merge operation.

3.5 Interpretation of Actuators

This section will give more insights into where dynamic links are from and why they are introduced. Then, the data arrangement for the dynamic links is presented.

The actuator can be considered a relationship constraint for a particular set of three joints. There are two types of actuators, rotary and linear. Each actuator corresponds to a dynamic link. For example, Figure 3.6 displays the two dynamic links for rotary and linear actuators in light red, respectively.

The dynamic link concept has been introduced for the example in Section 2.2. As discussed, it is an imaginary link that can be used for algebra. Furthermore, compared to the changing link length of a dynamic link, the unchanged distance constraint introduced by a binary link can be considered a *static binary link*.

A rotary actuator can be seen as a component that introduces an angle constraint and a distance constraint. The length of this dynamic link length can be computed according to the law of cosine, which is demonstrated in Equation 2.7.

Comparatively, a linear actuator is slightly simpler because it only introduces one changing distance constraint. Furthermore, the link length is the linear actuator input itself.

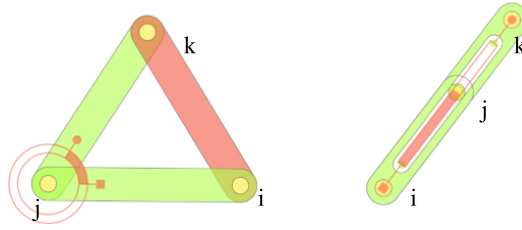


Figure 3.6: The letters denote the joints. For the rotary actuator (left), $l_{i,k}$ is the corresponding dynamic link. For the linear actuator (right), $l_{i,j}$ is the corresponding dynamic link.

In the position solving aspect, these dynamic links can be arranged with static binary links to form a closed-loop system of equations. The example in Section 2.3.1 shows the solution steps for Figure 2.3 and Step 3 used the dynamic link together with the other five static binary links to form the system of equations.

However, a dynamic link cannot completely replace its corresponding angle constraint. If the angle constraint is not taken into account in that example, then j_5 will have another solution that can be positioned above j_6 and j_4 . This is not trivial, but it is only a matter of how many finite numbers of solutions satisfy all these constraints. This finite number can go from one to some integer as long as all constraints can be satisfied. If not, then the number of solutions is zero, and this mechanism already hits the edge of the branch or circuit. Usually, the angle constraint serves as the additional information to determine which side of the solutions of the two sides is preferred (two sides for two 180 degrees, respectively). Therefore, it reduces half the number of solutions.

One can always stack more constraints than is necessary to get a finite number of solutions. That is, to solve the system of $2n$ variables (n joints) with more than $2n$ number of constraint equations. The example in Section 2.3.1 uses eight binary links (one of them is a dynamic link) and an angle constraint to solve the four joint positions. These nine constraints can be stacked together in a system of equations and be handed to Newton's solver. As long as these constraints are consistent, they can be satisfied altogether.

For rotary actuators, the joint positions need to satisfy the dynamic link constraints, regardless of whether or not these dynamic links are introduced into the system of equations. However, introducing dynamic links may help reduce the number of joints to compute per step, which is crucial in reducing computation time. The example in Section 2.3.1 shows that it is not necessary to solve all the three joints which together define the actuator. However, if this dynamic link is not introduced, four joints will need to be solved altogether with seven static binary links and one angle constraint. (Eight numbers of equations for eight unknown variables).

In summary, this chapter first reviews the components in the planar linkage mechanism simulation, then introduces a space-efficient method to denote a mechanism, following which is the reinterpretation of prismatic joints into revolute joints. Next, the interpretation of actuators is discussed, where the dynamic links can help reduce computation time.

Chapter 4

Mobility-based Analysis

This chapter introduces one of the crucial updates to MotionGen, which speeds up the simulation process by an apparent difference. The time efficiency is due to a better arrangement of the solving procedure, which is not touched as in-depth as this implementation.

In Chapter 2, the constraint equations for the planar linkage mechanism and the solvers to solve the system of equations formed by such constraints are introduced. Then, in Chapter 3, the topological representation and the data structure of these mechanisms are introduced. Finally, this chapter deals with the decomposition of the system of equations, which can reduce the overall simulation time.

A general pattern to solve the system of equations is that some unsolved variables (in other words, some unknown joints) are solved first. Then, these already-solved variables (in other words, the known joints) are substituted back into the system of equations to solve other unknown joints. Recursively doing these steps results in the solution of all joint positions in one single state. For one particular topological graph (in other words, the mechanism itself), the order of the set of joints to solve can be the same for all states. Furthermore, the solving procedure can be extracted through *graph traversal*.

Therefore, this chapter centers around the graph traversal and the properties found through this process. So, this chapter will present the study on mobility in Section 4.1, the algorithm for the extraction of solution steps in Section 4.2 and the estimation of time cost for the entire simulation process in Section 4.3. Finally, some specific cases are demonstrated in Section 4.4 and some statistics for the mechanisms presented in this thesis will be shown in Section 4.5.

4.1 Usage of the Mobility Formula

The degree-of-freedom (DOF), or *mobility*, of a multibody system is the number of parameters that define the configuration of the system. These parameters together build up the input for one particular state, which determines all the joint positions in the mechanism.

The Chebychev–Grübler–Kutzbach criterion, also known as the mobility formula, determines the DOF of a kinematic chain. Specifically, the mobility formula for planar rigid bodies is given as

$$dof = 3(n - 1) - 2 \cdot l - h, \quad (4.1)$$

where n is the number of links, l is the number of lower pairs (which takes away two DOF), and h is the number of higher pairs (which takes away one DOF).

In MotionGen, this formula is interpreted as demonstrated in Algorithm 7. In this algorithm, high pairs are not considered because there are no high pairs in the MotionGen simulator (all prismatic joints are converted into revolute joints). Furthermore, the rigid body collection, in other words, the link collection, is picked from the link-major order of B , i.e., part or whole of B_l .

Data: Links/Joint collections cJ

Result: degrees-of-freedom dof , common joints $jSet$, uncommon joints $jSetUC$

$jSet \leftarrow$ joints that are appeared in the cJ more than once;

$jSetUC \leftarrow$ joints that are appeared in the cJ only once;

$l \leftarrow 0$;

$n \leftarrow \text{len}(cJ)$;

foreach $joint \in jSet$ **do**

$l \leftarrow l + \text{number of rigid bodies that have this } joint - 1$;

end

$dof \leftarrow (n - 1) \cdot 3 - l \cdot 2$;

Algorithm 7: Degree-of-freedom calculation function, DOF

In positional analysis, for a well-defined linkage mechanism, the remaining DOF is zero when the actuator input is taken into account. Figure 4.1 shows an example.

As defined in Chapter 3, a link is a collection of joints and is a rigid body. The mobility formula can also be used to determine which links can

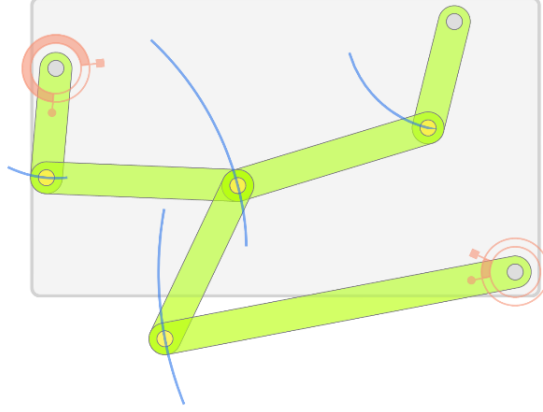


Figure 4.1: There are 7 rigid bodies and 7 rotary joints in this mechanism. One joint connects three rigid bodies while other joints connect two joints. The value of dof is $(7 - 1) \cdot 3 - (6 \cdot (2 - 1) + (3 - 1) \cdot 1) \cdot 2 = 2$. There are two actuators here, which together remove all remaining DOF. Therefore, this kinematic chain is properly constrained.

be merged into one. When the mobility is zero for a specific set of joints, the relative distance between any two joints in this set is unchanged. This means that some underlying binary links can be found during the process.

However, it is worth pointing out that there are two additional rules:

1. If two links share at least two joints, these two links can be merged into one.
2. The mobility formula is only sufficient when there is no partial over-constraint for a specific collection of links.

The first rule comes from the fact that there are distance constraints and angle constraints for the joints in a rigid body. Specifically, the shared two joints are the two reference joints and all other joints in the two links can be computed according to the angle and distance constraints. Additionally, the computation of joint positions can also go inversely for the other joints in the two links because they all have relative positions. Therefore, by definition, they are inside the same rigid body and the two links can be merged into one.

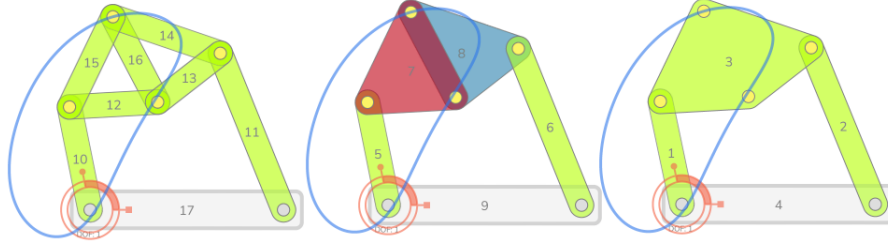


Figure 4.2: The mechanisms on the left and the middle can be converted to the mechanism on the right based on two different rules to merge links. The mechanism on the left will first merge l_{13} , l_{14} , l_{16} because the DOF for these three links is zero. Then this new link is merged with l_{12} and l_{15} because the DOF is again zero. For the mechanism in the middle, because l_7 and l_8 share two joints, these two links are merged into one. The resultant mechanism for the left and the middle ones is the one on the right. Therefore, these three mechanisms are all RRRR four-bar linkage mechanisms with two coupler points on the moving frame.

Furthermore, the DOF formula may return a negative value, which by the formula's definition is an over-constrained situation, which sometimes hints there is no solution. However, such a situation is always solvable, and the joints in these two links can always be solved when two of them are known. This is because any given three joints in these two links form a triangle whose side lengths are unchanged. To avoid the ambiguity of a negative value, we add the first rule as a special case of link merge. Figure 4.2 is an example where the formula and the first additional rules are used, respectively.

The second rule is applied because the formula is sufficient only when the over-constraint links are not taken into account. This problem can be illustrated by the two structures and their merged structure displayed in Figure 4.3 and Figure 4.4 respectively. They show that the mobility formula cannot directly apply to a complex kinematic chain. A more rigorous proof can be seen in [28].

With these additional rules, therefore, an accurate description of the DOF

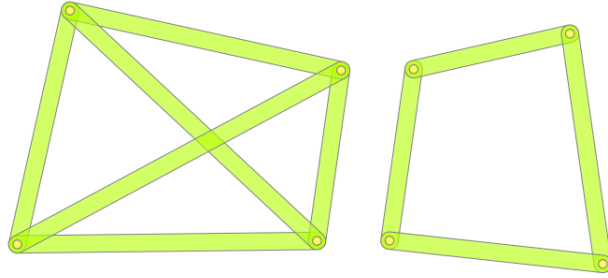


Figure 4.3: Left: over-constrained kinematic chain ($dof = (6 - 1) \cdot 3 - (4 \cdot (3 - 1)) \cdot 2 = -1$). Right: under-constrained kinematic chain ($dof = (4 - 1) \cdot 3 - (4 \cdot (2 - 1)) \cdot 2 = 1$)

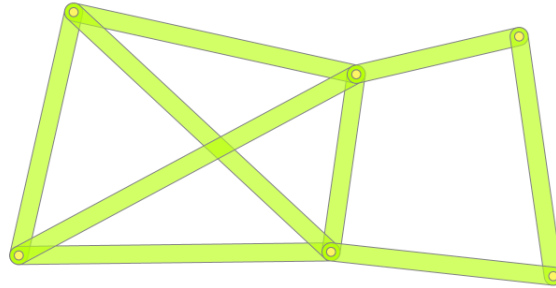


Figure 4.4: Partly over-constrained and partly under-constrained kinematic chain. ($dof = (9 - 1) \cdot 3 - ((4 - 1) \cdot 2 + (3 - 1) \cdot 2 + (2 - 1) \cdot 2) \cdot 2 = 0$)

formula can be summarized as follows.

$$dof = \begin{cases} < 0 & \text{At least one sub-kinematic chain is over-constrained.} \\ = 0 & \text{Undecided if its sub-kinematic chain is undecided;} \\ & \text{is properly constrained if each sub-kinematic chain is} \\ & \text{properly constrained.} \\ > 0 & \text{At least one sub-kinematic chain is under-constrained.} \end{cases}$$

To ensure the links are properly constrained, the sub-kinematic chain should be tested first. Based on the number of joints, the simplest kinematic chain is a binary link (2 joints), and the simplest closed-loop chain is two binary links where one is on top of the other. These two binary links can be merged according to the first additional rule. Then, for three joints, there can be three binary links; each collects two of the three, and every two links share only one joint. This closed-loop kinematic chain forms a triangle, resulting in the DOF within this chain being zero, i.e., the relative positions of these three joints do not change. When these three binary links are merged, the result is a ternary link.

The process of testing whether or not a collection of links can be merged into one starts from the smallest sub-collection of links. Then, in an order where the number of links in the sub-collection is non-decreasing, the mobility formula is used repeatedly, where some links can be merged during the process. Finally, when DOF is zero for the whole collection, one can conclude that all the links inside this collection can be merged into one. Additionally, only odd numbers of combinations of links are necessary to be stacked for the mobility formula because there are no high pairs in MotionGen, and if $(n - 1)$ is not even, then there is no way the mobility formula is equal to zero.

In the next section, the mobility formula and the two additional rules will be applied to two procedures.

4.2 Graph Traversal Algorithms

Some mechanism designers don't usually create mechanisms like the left and middle mechanisms in Figure 4.2 because they already know those two are equivalent to a four-bar RRRR linkage mechanism on the right in that Figure. More generally, they don't draw mergeable static links. However, MotionGen

is intended for all people. Some people may draw the mechanism chaotically. Finally, some may draw in a certain way for aesthetic concerns. Some may draw based on their practical uses of snappyXO kits, where most of them are binary static links.

Furthermore, if there are l number of binary links, a number of actuators, and n number of joints for a planar mechanism, then the simulation for one particular state is equivalent to solving a system of $(a + l)$ numbers of equations, for $2n$ numbers of variables. This set of equations can be solved by an optimization-based solver altogether. However, this is very time-inefficient. For complex mechanisms, the simulator can take more than one minute to solve for all states. Further details will be presented in Section 4.3. In this section, we use the mobility formula to do decomposition for this system of equations.

To handle the simulation efficiently, we introduced two graph traversal steps in the algorithm: the first step is to merge the static links to find more underlying binary links, which can help break the system of equations into smaller sections. The second step is merging binary links and dynamic links to find the solution procedure. They differ in that the first step is done only for static links and does not save the merging process. Then, the second step requires including the set of known joints, which collectively functions as a link. Because the known joints don't change their position, their relative distances don't change. Therefore, they form a rigid body. Furthermore, based on other joints that are solved beforehand, the joints that are newly introduced into this collection are the next joints to solve.

The pseudocodes in Algorithm 8 and Algorithm 9 summarizes the two steps respectively, where the MergeLink function is Algorithm 6 and the DOF function is Algorithm 7. *setComb* is a link major order format object. For example, if there are l_1 , l_2 and l_3 and l_1 has j_1 and j_2 , l_2 has j_2 and j_3 , l_3 has j_1 and j_3 , then this nested object *setComb* is

$$setComb = \begin{cases} 1 : & 1, 2; \\ 2 : & 2, 3; \\ 3 : & 1, 3; \end{cases}$$

Additionally, there is a *mLim* for Algorithm 8 to limit the maximum number of links to search because the total possible number of combinations is huge, which can take forever to do. Suppose this limitation is not introduced for the m number of links. In this case, the upper bound of the number of

Data: B_l (link-major format of B), L , T (J in its diagonal), $mLim$
Result: Updated B_l , L , T (J in its diagonal)
marker:
Remove duplicated links in B_l according to additional rule 1;
foreach $m \leftarrow$ odd numbers of (3 to $\min(\text{len}(B_l), mLim)$) **do**
 $setCombs \leftarrow$ combinations of picking m members of B_l ;
 foreach $setComb \subset setCombs$ **do**
 $dof, jSet, jSetUC \leftarrow \text{DOF}(setComb)$;
 if $dof \leq 0$ **then**
 get $linkIDs$ from $setComb$;
 $B_l, L, T \leftarrow \text{MergeLink}(B_l, L, T, linkIDs)$;
 go to marker;
 end
 end
end

Algorithm 8: Pseudocode for static link merge

combinations can be estimated with Equation 4.2, where the *odd* function returns the odd number for the given two values.

$$C_m^3 + C_m^5 + \dots C_m^{\text{odd}(m, m-1)} \leq \sum_{i=0}^m C_m^i = 2^m \quad (4.2)$$

Assuming the computation times for both Algorithm 6 and Algorithm 7 are all $O(m \cdot n)$ because the worst-case scenario is almost equivalent to a search for all elements in the B matrix whose size is $m \times n$, where n is the number of joints. Then, the overall upper bound of time cost would be $O(m \cdot n \cdot 2^m)$ because these two algorithms are done $O(2^m)$ times. In practice, we set $mLim = 3$ for the first step because other mergeable link collections are pretty rare. Furthermore, $mLim = 3$ results in an acceptable time of less than 10ms for common scenarios.

Then, comparatively, in Algorithm 9, the link merge operation is similar, but the link merge operation must include the ground link. In other words, in this algorithm, the ground link is gradually expanded until it includes every joint in the mechanism. Because this is expanded in such a manner, the time complexity for this algorithm is unavoidably $O(m \cdot n \cdot 2^m)$. Therefore, we introduce $tLim$ into the algorithm to limit the maximum time to search. If the complete search takes longer than $tLim$, the remaining unsolved joints

Data: $B_l, T, I, tLim$

Result: The solution process $steps$

$steps \leftarrow []$;

Extract ground link l_0 from B_l (Updated B_l does not have l_0);

Interpret I to get dynamic links and store these links in B_l ;

Start the timer and mark start time t_0 ;

marker:

foreach $m \leftarrow$ even numbers of (2 to $\min(\text{len}(B_l))$) **do**

$setCombs \leftarrow$ combinations of picking m members of B_l ;

foreach $setComb \subset setCombs$ **do**

$cJ \leftarrow \text{Stack}(setComb, l_0)$; $\backslash \backslash$ Stack the link collection

$dof, jSet, jSetUC \leftarrow \text{DOF}(cJ)$;

if $dof \leq 0$ **then**

if $\text{len}(jSet) == 1$ **then**

 Add solution step to $steps$ for the only joint in $jSet$ with the ArcSect solver;

 Mark constraints from I and T accordingly;

end

if $\text{len}(jSet) > 1$ **then**

 Add solution step to $steps$ for the joints in $jSet$, mark them to be solved with optimization solver;

 Mark constraints from I and T accordingly;

end

$l_0 \leftarrow \text{Union}(l_0, jSet)$; $\backslash \backslash$ Add new solved joints to ground

 Remove links in $setComb$ from B_l ;

if $\text{len}(jSetUC) > 0$ **then**

 Add solution step to $steps$ for each joint in $jSetUC$, mark them to be solved with the ArcSect solver;

 Mark constraints from I and T accordingly;

end

$l_0 \leftarrow \text{Union}(l_0, jSetUC)$;

go to marker;

end

 Get current time t_1 ;

 If $(t_1 - t_0) > tLim$, add solution step into $steps$, mark the remaining joints to be solved with the optimization solver;

 Mark constraints from I and T accordingly; Terminate search;

end

end

Algorithm 9: Pseudocode for process extraction

are packed into one to solve with an optimization solver. This will be further discussed in the next section.

Last but not least, both pseudocodes in Algorithm 8 and Algorithm 9 do not take space complexities into account, whose possible largest values can be $O(C_m^{\text{ceil}(\frac{m}{2})}) \leq O((\frac{e \cdot m}{\text{ceil}(m/2)})^{\text{ceil}(m/2)})$, where *ceil* is the ceiling function that returns the smallest ceiling integer for the given input. An approximation for the upper bound is $O(2.33^m)$, which is a huge value. However, the storage problem can be solved rather easily by sorting the combination and outputting one combination at a time.

4.3 Overall Algorithm for Simulation

Previous iterations of MotionGen [6, 7] has achieved partial graph traversal to decompose the system of equations for the ArcSect solver. However, they did not extract the solving procedure. Instead, the graph traversal procedure was redone for each input state, wasting time unnecessarily. Furthermore, when there were no more binary link pairs that could be used to solve for unknown joints, all remaining joints were handed to the NR-solver.

Comparatively, in this latest version, we not only extract the solving procedure and make the traversal be done only once, but we also extend the decomposition idea to more than two binary links. This overall simulation algorithm can be described with the pseudocode in Algorithm 10.

Here, the initialization of B , T , P_{init} are done with Algorithm 3, Algorithm 4 and Algorithm 5 respectively. Algorithm 6, Algorithm 7 are both used in Algorithm 8 and Algorithm 9. When there are more than one joint position to solve in one single step, either Algorithm 1 or Algorithm 2 is used.

The computation of *steps* and initialization of all matrices are the pre-simulation processes. Therefore, they can be seen as an entity of preparation in time cost analysis, so the remaining processes can be seen as an entity of simulation time. With all these considered, the overall time for simulation can be estimated according to Equation 4.3.

$$\text{Time} = \left(\sum_{i=0}^{\text{len}(\text{steps})} (2n_i)^{t_i} \cdot (a_i + l_i) \cdot C_i \right) \cdot s + \text{Prep-time} \quad (4.3)$$

Here, n_i denotes the number of joints to solve in step i . t_i is determined by the type of solver for the i -th step, shown in Equation 4.4. a_i and l_i

Data: *joints, links, slots, actuators* and some parameters
Result: Simulation results of a mechanism M
 Initialization for $B, T, P_{init}, J, L, S, I, V, P$;
 Reinterpret S, I and compute B_l from B ;
 Get updated B_l, L, T , from Algorithm 8;
 Get *steps* from Algorithm 9;
 Compute D according to T and P_{init} ;
foreach $input \subset V$ **do**
 foreach $step \subset steps$ **do**
 Solve for the joints in $step$ according to solver, D and $input$;
 If solvable, expand V and P accordingly;
 If not solvable, terminate the loops;
 end
end
 Stack V, P into M ;

Algorithm 10: Pseudocode for the overall process of simulation

are the number of actuators and the number of binary links to solve in the system of equations in step i , respectively. s is the number of states, and C_i is the estimated time constant for the solver used in step i . This constant is related to the solver's convergence speed and the time to stack all equations in this step. The ArcSect solver's constant is small because the error function converges in one step (analytical solution). The estimated constant for both Newton's methods is comparatively much more significant because they will stack more equations. Furthermore, the number of steps to converge depends on the positional difference of joints between this state and the previous state, which adds even more time.

$$t_i = \begin{cases} 1 & \text{If the ArcSect solver is used for the } i\text{-th step,} \\ 2 & \text{If an optimization based solver is used for the } i\text{-th step.} \end{cases} \quad (4.4)$$

By inspecting the overall time cost estimation expression, we can easily conclude that a complex mechanism generally takes more time to solve because it will stack more steps to the overall estimation expression than the simple ones and takes a longer time to do graph traversal. Furthermore, the overall time linearly increases with respect to the number of states.

Then, a second conclusion is that decomposing the system of equations can reduce the value for the first term. For each step that needs an optimization solver, $(a + l)$ is usually $O(n)$ when the constraint equations are picked properly. Therefore, for example, assume the time to compute with the Newton solver is comparable to n^3 times some constant C . If there are 30 joint positions to solve, then there are 60 variables to solve, and the time to solve by solving altogether is comparable to $60^3 C = 216,000C$. If we manage to find an arrangement of solutions where we can solve 10 of the joints at a time, then the time to solve these joints is comparable to $20^3 \times 3C = 24,000C$, which is about 10 times faster. It is even better if the number goes down even lower. If 5 of the joints are solved at a time, then the time is comparable to $10^3 \times 6C = 6,000C$, which is 36 times as fast as the speed of solving them altogether. Furthermore, in practice, it is uncommon to encounter a situation where at least 4 joints need to be solved altogether throughout the simulation, so the speed can be even faster for common cases.

However, at the same time, the graph traversal to find the solving procedure can be computationally expensive. The main portion of prep-time is mainly contributed by Algorithm 8 and Algorithm 9, which share a time complexity of $O(m \cdot n \cdot 2^m)$, as mentioned in Section 4.2. Although the graph traversal is done only once, in other words, the constant that multiplies with $m \cdot n \cdot 2^m$ is much less than $s \cdot C_i$, the solution procedure becomes very costly when there are lots of links. In the majority of the time, for most common and slightly complex mechanisms, this procedure does not exceed 10 ms. When the mechanism is created chaotically, it can take more than 50 ms, which hurts the design experience and is why we set a $tLim$ as the limit for search time in Algorithm 9. Some extreme cases we find during the application development will be demonstrated in Section 4.4.

Conclusively, there is a trade-off between a graph traversal procedure and the simulation time: either we find the least number of joints to solve per step, where the preparation can potentially take lots of time to search, or we solve them altogether for multiple states, which accumulatively can also take a considerable amount of time.

4.4 Case analysis

This section will go through several cases as subsections to discuss each aspect of this simulation package.

4.4.1 Floating Actuator

The latest MotionGen can do kinematic simulation regardless of whether the actuator is placed on the ground or not, and the actuator chain does not need to start from the ground. This section presents the simulation procedure for an RRRR mechanism. However, in one case, the actuator is placed on the ground joint, while in another case, the actuator is placed on a moving joint. These two cases are displayed in Figure 4.5. As demonstrated, both mechanisms can do full rotation because the actuator is placed on the shortest link and the shared four-bar linkage structure satisfies Grashof law.

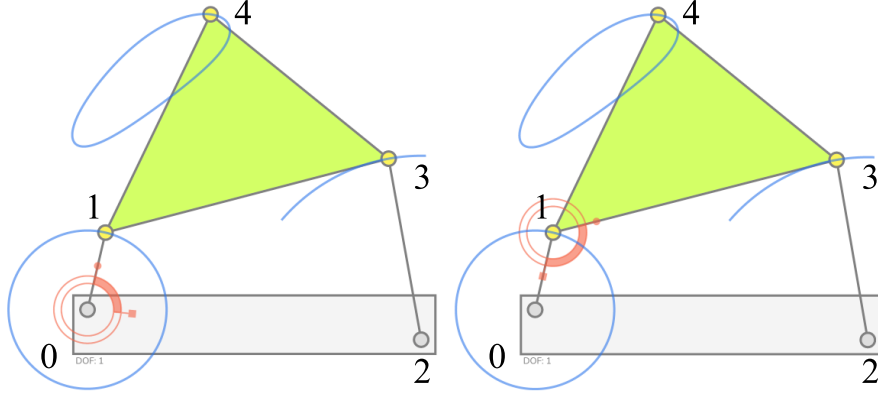


Figure 4.5: Left: RRRR mechanism whose actuator is fixed to ground; Right: RRRR mechanism whose actuator is fixed to a moving joint.

The solving procedure for the RRRR mechanism on the left is:

1. j_0 and j_2 are solved because they are the ground joints.
2. Using these two joints and $l_{0,2}$ and $l_{0,1}$, the dynamic link $l_{1,2}$ is computed according to the law of cosine and the rotary actuator input.
3. j_1 is solved with arc intersection of two circles: one's center is j_0 and its radius is $l_{0,1}$ and the other one's center is j_2 and its radius is $l_{1,2}$. Because there is an actuator, the solution should be picked according to the cross product value of $v_{0,2}$ and $v_{0,1}$.
4. j_3 is solved with the arc intersection of two circles: one's center is j_1 and its radius is $l_{1,3}$ and the other one's center is j_2 and its radius is

$l_{2,3}$. Because there is no actuator in the collection of joints (j_1, j_2, j_3) , the solution that is closest to the reference point is picked.

5. Similar to j_3 , j_4 is solved with arc intersection of two circles: one's center is j_3 and its radius is $l_{3,4}$ and the other one's center is j_1 and its radius is $l_{1,4}$.

The solution procedure for the RRRR mechanism on the right is:

1. j_0 and j_2 are solved because they are the ground joints.
2. Using $l_{1,0}$ and $l_{1,3}$, the dynamic link $l_{0,3}$ is computed according to the law of cosine and the rotary actuator input.
3. j_3 is solved with arc intersection of two circles: one's center is j_0 and its radius is $l_{0,3}$ and the other one's center is j_2 and its radius is $l_{2,3}$. Because there is no actuator in the collection of joints (j_0, j_2, j_3) , the solution that is closest to the reference point is picked.
4. j_1 is solved with arc intersection of two circles: one's center is j_0 and its radius is $l_{0,1}$ and the other one's center is j_3 and its radius is $l_{1,3}$. Because there is an actuator in the collection, the solution should be picked according to the cross product value of $v_{1,0}$ and $v_{1,3}$.
5. j_4 is solved with arc intersection of two circles: one's center is j_3 and its radius is $l_{3,4}$ and the other one's center is j_1 and its radius is $l_{1,4}$.

4.4.2 Multi-Slot

The current algorithm supports moving slots. Furthermore, a slot can be installed on other slots. An example is shown in Figure 4.6. For this mechanism, j_8 in the right figure shows that when the reinterpreted far joints are too close to the two end joints, the result is curved and cannot represent a straight path like the one on the left.

To solve this mechanism, these are the steps:

1. Convert slots to the combination of revolute joints and links like shown on the right in Figure 4.6. Each far joint forms a ternary link with two end joints. Each far joint then connects with the corresponding slot joint.

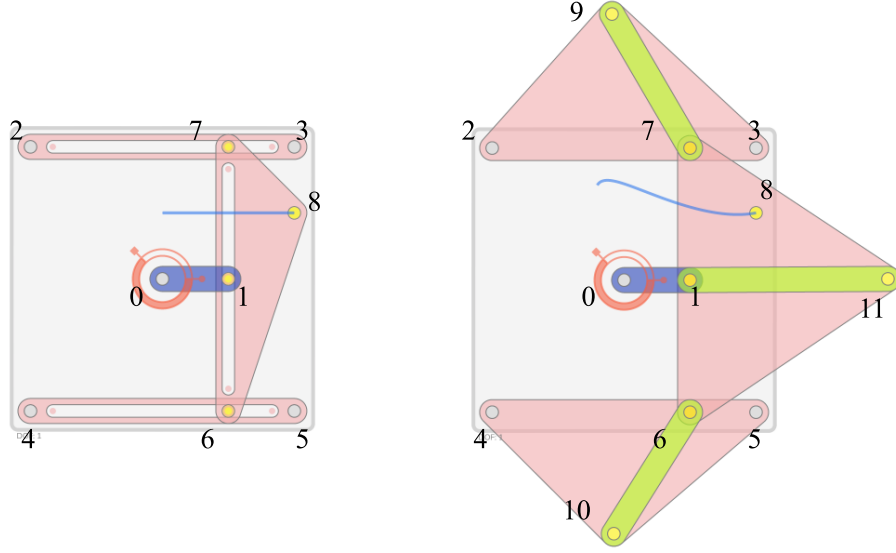


Figure 4.6: Left: The multi-slot mechanism, Right: the interpretation of such a mechanism. The constraining joints and constraining links are not displayed for visual clearness.

2. In the graph traversal phase, $j_0, j_2, j_3, j_4, j_5, j_9, j_{10}, j_{11}$ are ground joints and they are marked as solved.
3. j_1 is solved with the angle constraint from the actuator and fix joint j_0, j_2 (j_2 is another ground joint. An actuator needs three joints to define).
4. Through graph traversal, $l_{1,11}, l_{6,10}, l_{7,9}$ and the 4-ary link formed by j_6, j_7, j_8, j_{11} can be used to solve for j_6, j_7 and j_{11} altogether because they have zero DOF when they are connected to the ground link. This system has six equations to solve altogether.
5. Because j_6 and j_7 are solved and j_8 is in the same rigid body with j_6 and j_7 , j_8 can be solved with the use of arc intersection.
6. The remaining undisplayed constraining joints can be solved with arc intersection one by one because the position of slot joints and end joints were solved.

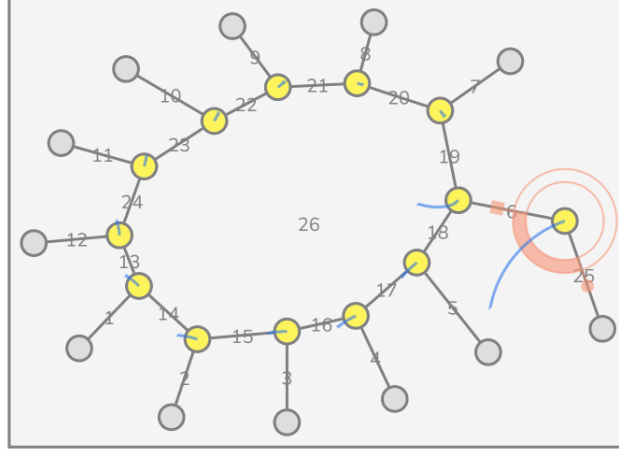


Figure 4.7: This millipede-like mechanism can only be solved by an optimization method and all movable joints have to be solved all at once. $l_1 \sim l_5$, l_6 together with the actuator and link l_{25} , and $l_7 \sim l_{12}$ are the “leg” links. Link $l_{13} \sim l_{24}$ are the “body” links. Link l_{26} is the ground link.

4.4.3 Extremely Complex Structure

Of all the extreme cases we found through experiment, one kind of kinematic structure is worth noting because, if a mechanism is assembled in accordance with such a pattern, all moving joints need to be solved simultaneously. Figure 4.7 is an example. In this millipede-like mechanism, there are 12 “legs” that touch the ground and 12 consecutive links that connect two legs one after another. In this case, all the distance constraints (links) and all the angle constraints (the actuator between l_6 and link l_{25}) need to be taken into the system of equations in order to solve for positions of movable joints. There are no ways to get around this, and this chapter’s algorithms cannot help.

Then, for this structure, if l_{25} and the actuator between l_6 and l_{25} are removed and the actuator joint is fixed onto the ground, all these moving links together with the ground link form a rigid body. In other words, all these joints are in one n -ary link. Based on this pattern, we created another structure in Figure 4.8. Although the structure is complex, this is a four-bar linkage mechanism. In fact, a moving 8-ary link is represented by multiple binary links.

This structure can be explained through analogy. in Figure 4.7, there are

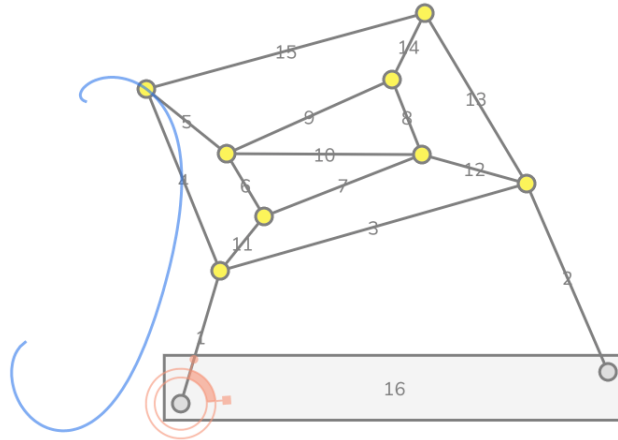


Figure 4.8: One of the static links of the millipede structure is on the four-bar linkage mechanism.

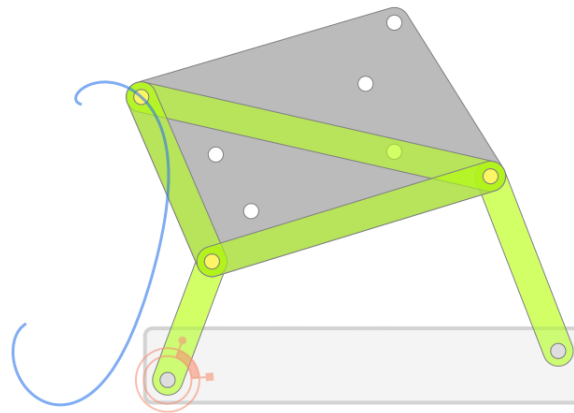


Figure 4.9: This is an equivalent representation for the mechanism for Figure 4.8. The binary links colored in green form a typical four-bar linkage mechanism and can be solved according to a similar pattern in Section 4.4.1.

12 “leg” links attached to the ground, compared to 4 “leg” links ($l_5, l_{11}, l_{12}, l_{14}$) in the structure in Figure 4.8. Then, in Figure 4.7, there are 12 “body” links, compared to 4 “body” links (l_3, l_4, l_{13}, l_{15}) in the moving frame in Figure 4.8. Next, the ground link l_{26} in Figure 4.7 is comparable to the rigid body formed by $l_6, l_7, l_8, l_9, l_{10}$ in Figure 4.8, which is because 1) l_8, l_9 and l_{10} form a ternary link and l_6, l_7 and l_{10} form another ternary link and 2) these two ternary links share two joints/a binary link, meaning these two are rigidly connected. Finally, such millipede-ground structure is a rigid body and $l_3 \sim l_{15}$ form an 8-ary link.

The fastest method to solve the coupler curve in blue for Figure 4.8 is using the binary links shown in Figure 4.9. If such an 8-ary link is not identified, six joints will need to be solved at once, using the distance constraints corresponding to $l_4 \sim l_{15}$ (Twelve binary links). However, if such an 8-ary link is identified, then this is not different from a four-bar linkage mechanism which can be solved with arc intersections purely, which requires two links at most at a time. However, this requires setting $mLim = 9$, which can be time-consuming.

4.4.4 Extremely Large Number of States

Pantographs are harmonic graphs that use electric motors and links to move a pen to create artistic drawings. Figure 4.10 is an example of such a machine. This is a rotating two-DOF five-bar mechanism with revolute joints only, where the path of j_5 is a long beautiful curve. This case is relatively simple in graph traversal. However, it has many states. Therefore, the simulation time is very long.

4.5 Structural Complexity Estimation

This section gives the statistic Table 4.1 for all the mechanisms mentioned in this thesis and uses the first term in Equation 4.3, which is

$$\sum_{i=0}^{len(steps)} (2n_i)^{t_i} \cdot (a_i + l_i) \cdot C_i,$$

to indicate the structural complexity of the mechanisms. The idea here is that the complexity of a mechanism relates to the system of equations to

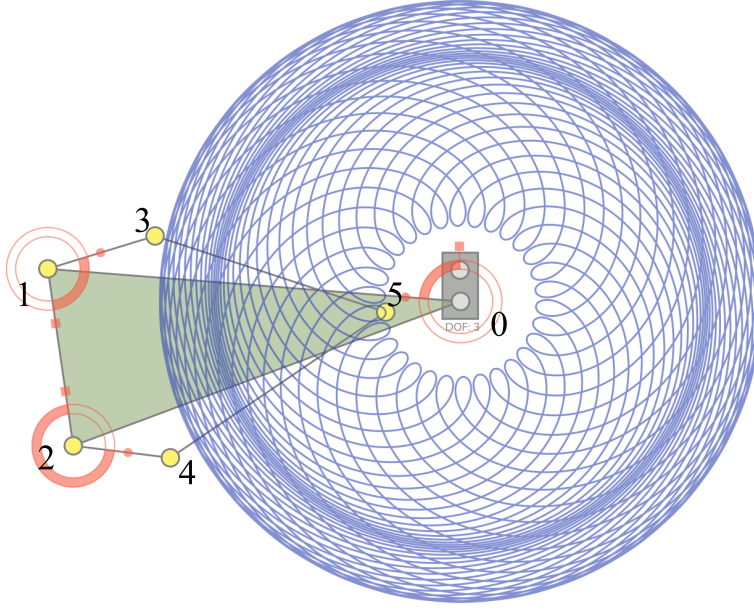


Figure 4.10: A pantograph machine

solve. For simplicity, the constant C_i is C_a for the ArcSect solver, and C_i is C_n for the Newton solvers (Regardless of which Newton method). This table also shows the actual time spent in simulation, which includes the graph traversal time.

From this table, it is straightforward to tell that when a mechanism can be solved with the arc intersection solver only, the time to compute is short, usually less than 10 ms. The outlier is the Pantograph machine, which has a comparatively much more number of states. If an optimization-based solver is used, the computation time is always more than 10 ms. Furthermore, because the millipede structure is inefficient, its path is not long. However, it takes more than 200 ms for this small number of states.

We also tested the speed difference between the ArcSect solver and Newton's solvers for the same question (system of two/three equations). For a fully rotating four-bar mechanism like Figure 3.1, the time to finish the simulation for both Newton solvers is around 30 ms, compared to approximately 3 ms for the ArcSect solver. For the pantograph machine in Figure 4.10, the GN-solver takes around 1,750 ms to finish, and the NR-solver takes around 1,450 ms to complete, compared to 60 ms for the ArcSect solver.

Mechanism	Estimated Complexity	Actual time spent in simulation
R robot arm in Figure 2.1	$6C_a$	Less than 2 ms
Mechanism in Figure 2.3	$6C_a + 216C_n$	~ 30 ms for both NR & GN
RRRR in Figure 3.1	$14C_a$	~ 3 ms
RRRR ,	$10C_a$	~ 4 ms
RRRP ,	$26C_a$	~ 4 ms
RRPR ,	$30C_a$	~ 4 ms
RRPP ,	$46C_a$	~ 4 ms
RPPR ,	$50C_a$	~ 3 ms
RPRP ,	$46C_a$	~ 4 ms
PRRP ,	$44C_a$	~ 5 ms
RPPP ,	$52C_a + 216C_n$	~ 10 ms for NR & GN
PRPP in Figure 3.4	$60C_a$	~ 5 ms
Mechanism in Figure 4.2	$18C_a$	~ 5 ms
Left Mechanism	$14C_a$	~ 4 ms
Right Mechanism	$14C_a$	~ 5 ms
in Figure 4.5		
Multislot Mechanism in Figure 4.6	$22C_a + 216C_n$	~ 30 ms for both NR & GN
Millipede in Figure 4.7	$17576C_n$	400 ms for NR 200 ms for GN
Mechanism in Figure 4.8	$10C_a + 1872C_n$ (Rigid body not found)	100 ms for NR 80 ms for GN
Pantograph mechine in Figure 4.10	$26C_a$	60 ms

Table 4.1: Table of statics of mechanisms

Chapter 5

Conclusion and Future Work

This thesis presents an efficient planar linkage simulation algorithm that improves upon the previous work. Major improvements include the repurposed simulation solvers, data structuring, and the extraction of solving procedures. In addition, multi-DOF kinematic simulation capabilities are expanded to allow for arbitrarily complex actuation profiles.

Future Work

There are several areas that this simulation package can improve. From the simulation aspect, there are five possible areas to search.

1. Dynamic analysis and dynamic simulation.
2. Introduction of other kinematic elements like gear/wheels. Specifically, wheel elements require velocity constraints (angle-angle) constraints. This type of constraint equation can be introduced in later implementations.
3. Efficient tree search for combination. This thesis presents the framework for graph traversal. However, it still has space to improve. One possible way to reduce search time is to truncate tree branches in the combination. The documentation for Mekin2d [24] presents some common kinematic structures.

4. Virtual constraint identification. Some mechanisms have singular properties that need to be detected. This will be a graph property detection problem.
5. Graph isomorphism detection algorithm. This helps to identify the types of mechanisms. This implementation introduces many graph theory techniques, which can serve as the foundation for these detections.
6. Branch defect problem. This is due to the placement of actuators. Additional works should focus on this part to draw the entire coupler curve.

Bibliography

- [1] McCarthy, J. M. and Soh, G. S., 2011, Geometric design of linkages, Springer.
- [2] Arthur Erdman, S. K., George Sandor, 2001, Mechanism Design: Analysis and Synthesis, Pearson; 4th edition.
- [3] Hunt, K. H., 1990, Kinematic geometry of mechanisms, Clarendon Press.
- [4] Hartenberg, R. S. and Denavit, J., 1964, Kinematic synthesis of linkages, McGraw-Hill.
- [5] Suh, C. H. and Radcliffe, C. W., 1978, Kinematics and Mechanism Design, John Wiley and Sons, New York.
- [6] Lodha, A., “Web-Based Design and Simulation of One- and Multi-Degrees of Freedom Planar n-Bar Linkages”, MS Thesis, Stony Brook University, USA, 2017.
- [7] Kothari, P., “Kinematic Simulation and Prototyping of N-Bar Planar Mechanisms using Web Technologies”, MS Thesis, Stony Brook University, USA, 2020.
- [8] Hain, K., 1967, Applied kinematics, McGraw Hill.
- [9] Dhingra, A. K., Almadi, A. N., and Kohli, D., 1999, “A Gröbner-Sylvester Hybrid Method for Closed-Form Displacement Analysis of Mechanisms”, Journal of Mechanical Design, **122**(4), pp. 431–438, doi: 10.1115/1.1290395, URL <http://dx.doi.org/10.1115/1.1290395>.
- [10] Wampler, C. W., 2000, “Solving the Kinematics of Planar Mechanisms by Dixon Determinant and a Complex-Plane Formulation”, Journal of

- Mechanical Design, **123(3)**, pp. 382–387, doi:10.1115/1.1372192, URL <http://dx.doi.org/10.1115/1.1372192>.
- [11] Wampler, C. W., 1999, “Solving the Kinematics of Planar Mechanisms”, Journal of Mechanical Design, **121(3)**, pp. 387–391, doi:10.1115/1.2829473, URL <http://dx.doi.org/10.1115/1.2829473>.
 - [12] Nielsen, J. and Roth, B., 1999, “Solving the Input/Output Problem for Planar Mechanisms”, Journal of Mechanical Design, **121(2)**, pp. 206–211, doi:10.1115/1.2829445, URL <http://dx.doi.org/10.1115/1.2829445>.
 - [13] Hernández, A. and Petuya, V., 2004, “Position analysis of planar mechanisms with R-pairs using a geometrical-iterative method”, Mechanism and Machine Theory, **39(2)**, pp. 133 – 152, doi:http://dx.doi.org/10.1016/S0094-114X(03)00110-1, URL <http://www.sciencedirect.com/science/article/pii/S0094114X03001101>.
 - [14] Javier García de Jalón, E. B., 2001, Kinematic and dynamic simulation of multibody systems-The Real-Time Challenge, Springer-Verlag.
 - [15] Ait-Aoudia, S., Jegou, R., and Michelucci, D., 2014, “Reduction of constraint systems”, URL <https://arxiv.org/abs/1405.6131>.
 - [16] Ait-Aoudia, S. and Foufou, S., 2010, “A 2D geometric constraint solver using a graph reduction method”, Advances in Engineering Software, **41(10)**, pp. 1187–1194, doi:https://doi.org/10.1016/j.advengsoft.2010.07.008, URL <https://www.sciencedirect.com/science/article/pii/S0965997810001006>.
 - [17] Bouma, W., Fudos, I., Hoffmann, C., Cai, J., and Paige, R., 1995, “Geometric constraint solver”, Computer-Aided Design, **27(6)**, pp. 487–501, doi:https://doi.org/10.1016/0010-4485(94)00013-4, URL <https://www.sciencedirect.com/science/article/pii/0010448594000134>.
 - [18] Fudos, I. and Hoffmann, C., 2004, “A Graph-Constructive Approach to Solving Systems of Geometric Constraints”, ACM Transactions on Graphics, **16**, doi:10.1145/248210.248223.
 - [19] Rector, D., “Linkage Software”, URL <http://blog.rectorsquid.com/linkage-mechanism-designer-and-simulator/>.

- [20] Campbell, M., “Planar Mechanism Kinematic Simulator”, URL <http://design.engr.oregonstate.edu/pmksintro.html>.
- [21] Trevor Dowd, H. Z., Robert Dutile, “Foundations for Category Theory”, URL https://web.wpi.edu/Pubs/E-project/Available/E-project-051820-072528/unrestricted/Final_PMKS-Enhancements.pdf, accessed: 7-23-2022.
- [22] Engineering, A., “SAM (Synthesis and Analysis of Mechanisms)”, URL <http://www.artas.nl/en>.
- [23] Petuya, V., Macho, E., Altuzarra, O., and Pinto, C., 2011, “Educational Software Tools for the Kinematic Analysis of Mechanisms”, *Comp. Appl. Eng. Education*, **6(4)**, pp. 261–266, URL www.ehu.es/compmech.
- [24] 2016, MeKin2D: Suite for Planar Mechanism Kinematics, volume Volume 5B: 40th Mechanisms and Robotics Conference of *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, doi:10.1115/DETC2016-59086, URL <https://doi.org/10.1115/DETC2016-59086>, v05BT07A083.
- [25] Moré, J. J., 1978, “The Levenberg-Marquardt algorithm: Implementation and theory”, G. A. Watson, ed., *Numerical Analysis*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 105–116.
- [26] Tsai, L.-W., 2001, *Mechanism design: Enumeration of kinematic structures according to function*, CRC Press.
- [27] Yamamoto, T., Iwatsuki, N., and Ikeda, I., 2020, “Automated Kinematic Analysis of Closed-Loop Planar Link Mechanisms”, *Machines*, **8(3)**, doi: 10.3390/machines8030041, URL <https://www.mdpi.com/2075-1702/8/3/41>.
- [28] Gogu, G., 2005, “Chebychev–Grübler–Kutzbach’s criterion for mobility calculation of multi-loop mechanisms revisited via theory of linear transformations”, *European Journal of Mechanics - A/Solids*, **24(3)**, pp. 427–441, doi:<https://doi.org/10.1016/j.euromechsol.2004.12.003>, URL <https://www.sciencedirect.com/science/article/pii/S0997753805000033>.